Notes on Deep Hashing

Jianglin Lu

2020.04.07

2013

2013 NIPS The Power of Asymmetry in Binary Hashing

Behnam Neyshabur, Payman Yadollahpour, Yury Makarychev, Ruslan Salakhutdinov, Nathan Srebro (without code)

之前方法的缺点：

Similarity S(x, x') between two objects is approximated by the hamming distance between the outputs of **the same hash function**, i.e. **between f(x) and f(x'),** for some f \in {+-1}^k. The emphasis here is that **the same hash function is applied to both x and x',** (in methods like LSH multiple hashes might be used to boost accuracy, but the comparison is still between outputs of the same function).

狭义的非对称: 一个用实值一个用二值

The only exception we are aware of is where a single mapping of objects to fractional vectors $\tilde{f}(x) \in [-1,1]^k$ is used, its thresholding $f(x) = \operatorname{sign} \tilde{f}(x) \in \{\pm 1\}^k$ is used in the database, and similarity between x and x' is approximated using $\left\langle f(x), \tilde{f}(x') \right\rangle$. This has become known as "asymmetric hashing" [2, 4], but even with such a-symmetry, both mappings are based on the same fractional mapping $\tilde{f}(\cdot)$. That is, the asymmetry is in that one side of the comparison gets thresholded while the other is fractional, but not in the actual mapping.

非对称方法的优势：

Approximating the similarity between x and x' as the hamming distance between f(x) and g(x'), for two distinct binary codes f, g, rather than as the hamming distance between f(x) and f(x').

Asymmetric hashes can be **used in almost all places** where symmetric hashes are typically used, usually **without any additional storage or computational cost.**

Using an **asymmetric hash** means **using different hash functions for the database and for the query.** This **neither increases the size of the database representation, nor the computational or communication cost of populating the database** or performing a query, as the exact same operations are required.

We argue that using **two different hash functions to encode database objects and queries allows for much more flexibility in choosing the database hash.** Unlike the query hash, **which has to be stored compactly and efficiently** evaluated on queries as they appear, if the database is fixed, **an arbitrary mapping of database objects to bit strings may be used.** We demonstrate that this can indeed **increase similarity**

accuracy while reducing the bit length required.

重点: **<u, v> = (k – 2d$_h$ (u, v)), u, v \in {+-1}^k.**

"neighbors". A symmetric binary coding of $\mathcal{X}$ is a mapping $f : \mathcal{X} \to \{\pm 1\}^k$, where k is the bit-length of the code. We are interested in constructing codes such that the hamming distance between $f(x)$ and $f(x')$ corresponds to the similarity $S(x, x')$. That is, for some threshold $\theta \in \mathbb{R}$, $S(x, x') \approx \text{sign}(\langle f(x), f(x')\rangle - \theta)$. Although discussing the hamming distance, it is more convenient for us to work with the inner product $\langle u, v\rangle$, which is equivalent to the hamming distance $d_h(u, v)$ since $\langle u, v\rangle = (k - 2d_h(u, v))$ for $u, v \in \{\pm 1\}^k$.

广义的非对称: 用两个不同的哈希函数

In this paper, we propose using two *distinct* mappings $f(x), g(x) \in \{\pm 1\}^k$ and approximating the similarity $S(x, x')$ by the hamming distance between $f(x)$ and $g(x')$. We refer to such hashing schemes as "asymmetric". Our main result is that even if the target similarity function is symmetric and "well behaved" (e.g., even if it is based on Euclidean distances between objects), using asymmetric binary hashes can be much more powerful, and allow better approximation of the target similarity with shorter code lengths. In particular, we show extreme examples of collections of points in Euclidean space, where the neighborhood similarity $S(x, x')$ can be realized using an asymmetric binary hash (based on a pair of distinct functions) of length $O(r)$ bits, but where a symmetric hash (based on a single function) would require at least $\Omega(2^r)$ bits. Although actual data is not as extreme, our experimental results on real data sets demonstrate significant benefits from using asymmetric binary hashes.

Denoting $U \in \{\pm 1\}^{n\times k}$ for the matrix whose columns contain the codewords u_i, the minimal binary code length that allows exactly representing S is then given by the following matrix factorization problem:

$$k_s(S) = \min_{k,U,\theta} k \quad \text{s.t} \quad \begin{array}{ll} U \in \{\pm 1\}^{k\times n} & \theta \in \mathbb{R} \\ Y \triangleq U^\top U - \theta \mathbb{1}_n & \\ \forall_{ij}\ S_{ij} Y_{ij} > 0 & \end{array} \tag{1}$$

where $\mathbb{1}_n$ is an $n \times n$ matrix of ones.

We begin demonstrating the power of asymmetry by considering an *asymmetric* variant of the above problem. That is, even if S is symmetric, we allow associating with each object x_i two distinct binary codewords, $u_i \in \{\pm 1\}^k$ and $v_i \in \{\pm 1\}^k$ (we can think of this as having two arbitrary mappings $u_i = f(x_i)$ and $v_i = g(x_i)$), such that $S_{ij} = \text{sign}(\langle u_i, v_j\rangle - \theta)$. The minimal asymmetric binary code length is then given by:

$$k_a(S) = \min_{k,U,V,\theta} k \quad \text{s.t} \quad \begin{array}{ll} U, V \in \{\pm 1\}^{k\times n} & \theta \in \mathbb{R} \\ Y \triangleq U^\top V - \theta \mathbb{1}_n & \\ \forall_{ij}\ S_{ij} Y_{ij} > 0 & \end{array} \tag{2}$$

Writing the binary coding problems as matrix factorization problems is useful for understanding the power we can get by asymmetry: even if S is symmetric, and even if we seek a symmetric Y, insisting on writing Y as a square of a binary matrix might be a tough constraint. This is captured in the following Theorem, which establishes that there could be an exponential gap between the minimal asymmetry binary code length and the minimal symmetric code length, even if the matrix S is symmetric and very well behaved:

定理 1 暂未看明白....

结论:

There exists data sets for which **an asymmetric binary hash might be much shorter than a symmetric hash. Asymmetric hashes could be much more powerful,** and should prompt us to consider them instead of symmetric hashes.

Asymmetric binary codes can allow for better approximations using shorter codes, so it is natural to seek asymmetric codes **for generalization.**

This form of asymmetry can allow us for **greater approximation power**, and thus **better accuracy with shorter codes**, at **no additional computational or storage cost**.
对于新的样本使用非对称哈希函数:

The symmetric binary hashing approach (e.g. [8]), would be to find a single parametric mapping $f : \mathcal{X} \to \{\pm 1\}^k$ such that $S(x, x_i) \approx \text{sign}(\langle f(x), f(x_i)\rangle - \theta)$ for future queries x and database objects x_i, calculate $f(x_i)$ for all database objects x_i, and store these hashes (perhaps in a hash table allowing for fast retrieval of codes within a short hamming distance). The asymmetric approach described above would be to find two parametric mappings $f : \mathcal{X} \to \{\pm 1\}^k$ and $g : \mathcal{X} \to \{\pm 1\}^k$ such that $S(x, x_i) \approx \text{sign}(\langle f(x), g(x_i)\rangle - \theta)$, and then calculate and store $g(x_i)$.

可学习的单词:

Later work suggested learning hash functions **attuned** to the distribution of the data.

Attuned: 熟悉; 适应; 习惯; 使协调; 使合拍; 为…调音; 调合波长

可学习的例句:

Indeed, it only takes a fraction of a second to retrieve a shortlist of similar items from a corpus containing billions of data points. Compact binary codes are **remarkably storage efficient,** and allow one to store massive datasets in memory.

The computational and communication costs **scale linearly with the length of the hash**, and more importantly, **the memory cost of the hash table can scale exponentially with the length.**

Then, given a new query object, we would like to **efficiently compute a compact binary code for a given query** and retrieve similar items in the database very fast by **finding binary codes in the database that are within small hamming distance from the query binary code.**

Recall that it is important to ensure that **the bit length of the hashes are small, as short codes allow for very fast hamming distance calculations and low communication costs** if the codes need to be sent remotely. More importantly, if we would like to **store the database in a hash table allowing immediate lookup, the size of the hash table is exponential in the code length.**

Using asymmetric codes instead of symmetric codes could be much more powerful, and **allow for shorter and more accurate codes**, and is usually straightforward and **does not require any additional computational, communication or significant additional memory resources** when using the code. We would therefore encourage the use of such **asymmetric codes (with two distinct hash mappings)** wherever binary hashing is used to approximate similarity.

2014

2014 ACM SIGIR Supervised Hashing with Latent Factor Models

Peichao Zhang, Wei Zhang, Wu-Jun Li, Minyi Guo (with MATLAB code)

之前方法的缺点：

Semantic gap的定义及其所带来的问题(无监督方法的缺点)：

High level semantic description of an object often **differs from** the extracted **low level feature descriptors**, known as **semantic gap**, returning nearest neighbors **according to metric distances** between the feature vectors **doesn't always guarantee a good search quality.**

However, some existing supervised methods, like MLH, suffer from **a very large amount of training time**, making it difficult to apply to large-scale datasets.

LFH方法的优势：

Based on latent factor models, **the likelihood of the pairwise similarities are elegantly modeled as a function of the Hamming distance** between the corresponding data points.

可学习的例句：

使用二值码的好处：

The **advantage of binary codes representation** over the original feature vector representation is **twofold**. Firstly, **each dimension of a binary code can be stored using only 1 bit while several bytes are typically required for one dimension of the original feature vector, leading to a dramatic reduction in storage cost.** Secondly, by using binary codes representation, **all the data points within a specific Hamming distance to a given query can be retrieved in constant or sub-linear time regardless of the total size of the dataset.**

The similarity labels **S can be constructed from the neighborhood structure by thresholding on the metric distances** between the feature vectors. However, **such S is of low quality since no semantic information is utilized**. In supervised hashing setting, **S is often constructed from the semantic labels** within the data points. **Such labels are often built with manual effort to ensure its quality.**

Let Θ_{ij} denote half of the inner product between two binary codes $\mathbf{b}_i, \mathbf{b}_j \in \{-1,1\}^Q$:

$$\Theta_{ij} = \frac{1}{2}\mathbf{b}_i^T\mathbf{b}_j.$$

The likelihood of the observed similarity labels $\mathcal{S}$ can be defined as follows:

$$p(\mathcal{S} \mid \mathbf{B}) = \prod_{s_{ij}\in\mathcal{S}} p(s_{ij} \mid \mathbf{B}), \tag{1}$$

with

$$p(s_{ij} \mid \mathbf{B}) = \begin{cases} a_{ij}, & s_{ij} = 1 \\ 1 - a_{ij}, & s_{ij} = 0 \end{cases},$$

where $a_{ij} = \sigma(\Theta_{ij})$ with σ being the logistic function:

$$\sigma(x) = \frac{1}{1+e^{-x}}.$$

It is easy to prove the following relationship between the Hamming distance $\mathrm{dist}_H(\cdot,\cdot)$ and the inner product of two binary codes:

$$\mathrm{dist}_H(\mathbf{b}_i, \mathbf{b}_j) = \frac{1}{2}(Q - \mathbf{b}_i^T\mathbf{b}_j) = \frac{1}{2}(Q - 2\Theta_{ij}).$$

We can find that the smaller the $\mathrm{dist}_H(\mathbf{b}_i, \mathbf{b}_j)$ is, the larger $p(s_{ij} = 1 \mid \mathbf{B})$ will be. Maximizing the likelihood of $\mathcal{S}$ in (1) will make the Hamming distance between two similar points as small as possible, and that between two dissimilar points as high as possible. Hence this model is reasonable and matches the goal to preserve similarity.

With some prior $p(\mathbf{B})$, the posteriori of $\mathbf{B}$ can be computed as follows:

$$p(\mathbf{B} \mid \mathcal{S}) \sim p(\mathcal{S} \mid \mathbf{B})p(\mathbf{B}).$$

We can use maximum a posteriori estimation to learn the optimal $\mathbf{B}$. However, directly optimizing on $\mathbf{B}$ is an NP-hard problem [34]. Following most existing hashing methods, we compute the optimal $\mathbf{B}$ through two stages. In the first stage, we relax $\mathbf{B}$ to be a real valued matrix $\mathbf{U} \in \mathbb{R}^{N \times Q}$. The ith row of $\mathbf{U}$ is called the latent factor for the ith data point. We learn an optimal $\mathbf{U}$ under the same probabilistic framework as for $\mathbf{B}$. Then in the second stage, we perform some rounding technique on the real valued $\mathbf{U}$ to get the binary codes $\mathbf{B}$.

More specifically, we replace all the occurrences of $\mathbf{B}$ in previous equations with $\mathbf{U}$. Θ_{ij} is then re-defined as:

$$\Theta_{ij} = \frac{1}{2}\mathbf{U}_{i*}\mathbf{U}_{j*}^T.$$

Similarly, $p(\mathcal{S} \mid \mathbf{B})$, $p(\mathbf{B})$, $p(\mathbf{B} \mid \mathcal{S})$ are replaced with $p(\mathcal{S} \mid \mathbf{U})$, $p(\mathbf{U})$, $p(\mathbf{U} \mid \mathcal{S})$, respectively. We put a normal distribution on $p(\mathbf{U})$:

$$p(\mathbf{U}) = \prod_{d=1}^{Q} \mathcal{N}(\mathbf{U}_{*d} \mid \mathbf{0}, \beta\mathbf{I}),$$

where $\mathcal{N}(\cdot)$ denotes the normal distribution, $\mathbf{I}$ is an identity matrix, and β is a hyper-parameter. The log posteriori of $\mathbf{U}$ can then be derived as:

$$\begin{aligned} L &= \log p(\mathbf{U} \mid \mathcal{S}) \\ &= \sum_{s_{ij} \in \mathcal{S}} (s_{ij}\Theta_{ij} - \log(1 + e^{\Theta_{ij}})) - \frac{1}{2\beta}\|\mathbf{U}\|_F^2 + c, \end{aligned} \tag{2}$$

where $\|\cdot\|_F$ denotes the Frobenius norm of a matrix, and c is a constant term independent of $\mathbf{U}$. The next step is to learn the optimal $\mathbf{U}$ that maximizes L in (2).

2.3.2 *Out-of-Sample Extension*

In order to perform ANN search, we need to compute the binary code $\mathbf{b}$ for a query $\mathbf{x}$ which is not in the training set. We achieve this by finding a matrix $\mathbf{W} \in \mathbb{R}^{D \times Q}$ that maps $\mathbf{x}$ to $\mathbf{u}$ in the following way:

$$\mathbf{u} = \mathbf{W}^T\mathbf{x}.$$

We then perform the same rounding technique discussed in Section 2.3.1 on $\mathbf{u}$ to obtain $\mathbf{b}$.

We use linear regression to train $\mathbf{W}$ over the training set. The squared loss with regularization term is shown below:

$$L_e = \|\mathbf{U} - \mathbf{X}\mathbf{W}\|_F^2 + \lambda_e\|\mathbf{W}\|_F^2.$$

And the optimal $\mathbf{W}$ can be computed as:

$$\mathbf{W} = (\mathbf{X}^T\mathbf{X} + \lambda_e\mathbf{I})^{-1}\mathbf{X}^T\mathbf{U}. \tag{5}$$

2014 AAAI Supervised Hashing for Image Retrieval via Image Representation Learning

Rongkai Xia, Yan Pan, Hanjiang Lai, Cong Liu, Shuicheng Yan (without code)

之前方法的缺点：(没有细推第一步求H的过程)

However, such **hand-crafted visual features**, which are extracted in an unsupervised fashion, **do not necessarily guarantee to accurately preserve the semantic similarities** of image pairs, i.e., **a pair of semantically similar/dissimilar images may not have feature vectors with relatively small/large Euclidean distance.**

Semantic Hashing is a hashing method which automatically constructs binary-code feature representation for images by **a multi-layer auto-encoder, with the raw pixels of images being directly used as input**. However, **Semantic Hashing imposes difficult optimization** and **its performance was surpassed by recently proposed hashing methods.**

However, **LSH** methods usually **require long hash codes** to achieve satisfactory search accuracies, resulting in **large space for storage and low recall in search.**

Most of the existing hashing methods for images **first encode each input image by hand-crafted visual features**, which may often **degrade their hashing**

performance because **hand-crafted features do not necessarily capture accurate similarity of the images.**

CNNH方法的优势：

We propose to **simultaneously learn a good feature representation** for the input images as well as **a set of hash functions**, via **a deep convolutional network tailored to the learned hash codes in H** and optionally the discrete class labels of the images.

CNNH **simultaneously learns** a set of **hashing functions** as well as a useful **image representation** tailored to the hashing task.

The proposed method has two advantages: (1) We propose to **solve the approximate hash code learning problem by a coordinate descent method**, which is empirically **much faster** than the corresponding decomposition method used in TSH. (2) In contrast to the hand-crafted features used in TSH, we propose to **learn hash functions while simultaneously training a useful feature representation** for the input images.

The proposed **coordinate descent algorithm is much faster** to converge than the BCD algorithm.

可学习的例句：

A key question in learning-based hashing for images is **how to encode images into a useful feature representation** so as to enhance the hashing performance.

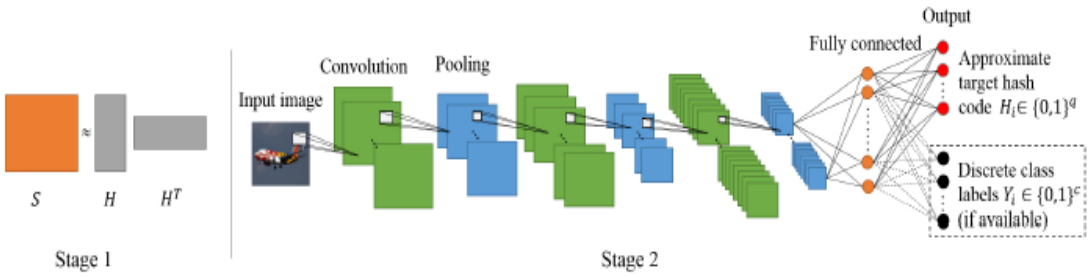
One of the popular ways is **decomposing the learning process into a hash code learning stage followed by a hash function learning stage.**

实验设置：

Since CNNH and CNNH+ use **random initialization of H** in stage 1, in all of the experiments, the results of CNNH and CNNH+ are **the average of 5 trials.**

$$\min_{H} \sum_{i=1}^{n} \sum_{j=1}^{n} (S_{ij} - \frac{1}{q} H_{i\cdot} H_{j\cdot}^{T})^2 = \min_{H} ||S - \frac{1}{q} H H^T||_F^2, \quad (2)$$

$$\min_{H} ||S - \frac{1}{q} H H^T||_F^2 \ \ s.t.\ H \in [-1, 1]^{n \times q}. \quad (3)$$



2015

2015 CVPR Simultaneous Feature Learning and Hash Coding with Deep Neural Networks

Hanjiang Lai, Yan Pan, Ye Liu , Shuicheng Yan (with Caffe code)

之前方法的缺点：

However, such **fixed hand-crafted visual features may not be optimally compatible with the coding process**. In other words, **a pair of semantically similar/dissimilar images may not have feature vectors with relatively small/large Euclidean distance.**

However, in this two-stage method (CNNH), the learned approximate hash codes are used to guide the learning of the image representation, but **the learned image representation cannot give feedback for learning better approximate hash codes**. **This one-way interaction thus still has limitations.**

However, a limitation in CNNH is that the **learned image representation cannot be used to improve the learning of approximate hash codes**, although the learned approximate hash codes can be used to guide the learning of image representation.

In most of the existing supervised hashing methods, the **side information is in the form of pairwise labels** that indicate the semantical similarities/dissimilarities on image pairs.

NINH方法的优势：

We learn the image representation and the hash codes **in one stage**, such that these **two tasks have interaction and help each other forward.**

Some parts of the proposed architecture are designed on the basis of [11] that uses additional 1*1 convolution layers to **increase the representational power of the networks** (use NIN network).

Learn hash functions that preserve relative similarities of the form "**image I is more similar to image I+ than to image I-**". Such a form of **triplet-based relative similarities** can be **more easily obtained** than pairwise similarities (e.g., the click-through data from image retrieval systems). Furthermore, given the side information of pairwise similarities, one can easily generate a set of triplet constraints.

可学习的例句：

Specifically, in the fully-connected alternative in Figure 2(b), **each hash bit is generated on the basis of the whole (and the same) input image feature vector,** which may **inevitably result in redundancy among the hash bits.** On the other hand, since each hash bit is generated from **a separated slice of features,** the output hash codes from the proposed divide-and-encode module may **be less redundant to each other.** Hash codes with fewer redundant bits **are advocated by** some recent research.

Batch-Orthogonal Locality Sensitive Hashing theoretically and empirically shows

that **hash codes generated by batch-orthogonalized random projections are superior to those generated by simple random projections, where batch-orthogonalized projections generate fewer redundant hash bits than random projections.**

The underlying reason for the improvement may be that, compared to the FC alternative, the output hash codes from the divide-and-encode modules are less redundant to each other.

The underlying reason why the architecture with a shared sub-network performs better than the one with two independent sub-networks **may be that** the **training samples are not enough for networks with too much parameters** (e.g., 500 training images per class on CIFAR-10 and NUS-WIDE).

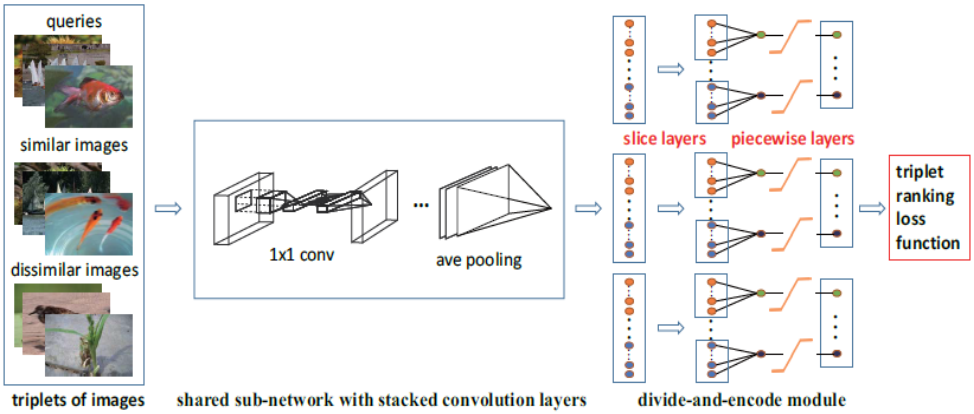


Figure 1. Overview of the proposed deep architecture for hashing. The input to the proposed architecture is in the form of triplets, i.e., (I, I^+, I^-) with a query image I being more similar to an image I^+ than to another image I^-. Through the proposed architecture, the image triplets are first encoded into a triplet of image feature vectors by a shared stack of multiple convolution layers. Then, each image feature vector in the triplet is converted to a hash code by a divide-and-encode module. After that, these hash codes are used in a triplet ranking loss that aims to preserve relative similarities on images.

This architecture accepts input images in **a triplet form**. Given triplets of input images, the pipeline of the proposed architecture contains **three parts**: 1) a sub-network with multiple convolution-pooling layers to capture a representation of images; 2) a divide-and-encode module designed to generate bitwise hash codes; 3) a triplet ranking loss layer for learning good similarity measures.

Table 1. Configurations of the shared sub-network for input images of size 256 × 256

type	filter size/stride	output size
convolution	11× 11 / 4	96 × 54 × 54
convolution	1× 1 / 1	96 × 54 × 54
max pool	3× 3 / 2	96 × 27 × 27
convolution	5× 5 / 2	256 × 27 × 27
convolution	1× 1 / 1	256 × 27 × 27
max pool	3× 3 / 2	256 × 13 × 13
convolution	3× 3 / 1	384 × 13 × 13
convolution	1× 1 / 1	384 × 13 × 13
max pool	3× 3 / 2	384 × 6 × 6
convolution	3× 3 / 1	1024 × 6 × 6
convolution	1 × 1 / 1	(50 × # bits) × 6 × 6
ave pool	6× 6 / 1	(50 × # bits) × 1 × 1

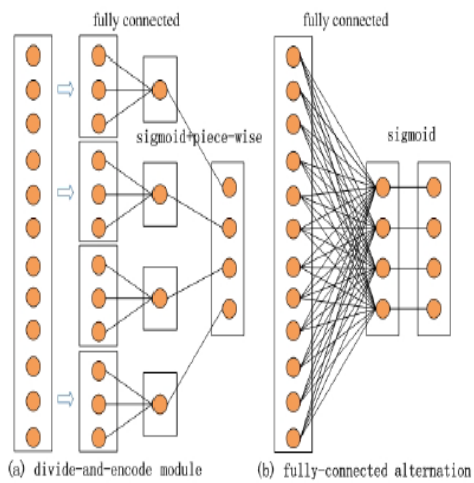


Figure 2. (a) A divide-and-encode module. (b) An alternative that consists of a fully-connected layer, followed by a sigmoid layer.

$$\begin{aligned}&\hat{\ell}_{triplet}(\mathcal{F}(I),\mathcal{F}(I^+),\mathcal{F}(I^-))\\&=\max(0,1-(||\mathcal{F}(I)-\mathcal{F}(I^-)||_{\mathcal{H}}-||\mathcal{F}(I)-\mathcal{F}(I^+)||_{\mathcal{H}}))\\&s.t.\ \mathcal{F}(I),\ \mathcal{F}(I^+),\ \mathcal{F}(I^-)\in\{0,1\}^q,\end{aligned}\tag{1}$$

$$\begin{aligned}&\ell_{triplet}(\mathcal{F}(I),\mathcal{F}(I^+),\mathcal{F}(I^-))\\&=\max(0,||\mathcal{F}(I)-\mathcal{F}(I^+)||_2^2-||\mathcal{F}(I)-\mathcal{F}(I^-)||_2^2+1)\\&s.t.\ \mathcal{F}(I),\ \mathcal{F}(I^+),\ \mathcal{F}(I^-)\in[0,1]^q.\end{aligned}\tag{2}$$

This variant of triplet ranking loss is convex. Its (sub-)gradients with respect to $\mathcal{F}(I)$, $\mathcal{F}(I^+)$ or $\mathcal{F}(I^-)$ are

$$\begin{aligned}\frac{\partial\ell}{\partial b}&=(2b^- - 2b^+)\times I_{||b-b^+||_2^2-||b-b^-||_2^2+1>0}\\\frac{\partial\ell}{\partial b^+}&=(2b^+ - 2b)\times I_{||b-b^+||_2^2-||b-b^-||_2^2+1>0}\\\frac{\partial\ell}{\partial b^-}&=(2b^- - 2b)\times I_{||b-b^+||_2^2-||b-b^-||_2^2+1>0},\end{aligned}\tag{3}$$

where we denote $\mathcal{F}(I)$, $\mathcal{F}(I^+)$, $\mathcal{F}(I^-)$ as b, b^+, b^-. The indicator function $I_{condition}=1$ if $condition$ is true; otherwise $I_{condition}=0$. Hence, the loss function in (2) can be easily integrated in back propagation in neural networks.

2015 CVPR Deep Hashing for Compact Binary Codes Learning

Venice Erin Liong, Jiwen Lu, Gang Wang, Pierre Moulin, Jie Zhou (without code)

之前方法的缺点：

However, most these **data-dependent hashing methods cannot well capture the nonlinear manifold structure of samples.**

While some **kernel-based hashing** methods have been presented, they still **suffer from the scalability problem** because these kernel-based methods **cannot obtain the explicit nonlinear mapping.**

While these hashing methods have achieved reasonably good performance in many applications, **most of them usually seek a single linear projection, which cannot well capture the nonlinear structure of samples.**

While deep learning has achieved great success in various visual application, **little progress of deep learning have been made in hashing-based large scale visual search.**

To our knowledge, **semantic hashing is the first work on using deep learning techniques for hashing.** They applied the **stacked Restricted Boltzmann Machine (RBM) learn compact binary codes** for visual search. However, **their model is complex and requires pre-training,** which is not efficient for practical applications.

SDH方法的优势：

Unlike existing binary codes learning methods which seek a single linear projection to map each sample into a binary vector, we **develop a deep neural network** to seek **multiple hierarchical non-linear transformations** to learn these binary codes.

Our model is learned under three constraints at the top layer of the deep network: 1) **the loss between the original feature descriptor and the learned binary vector is minimized**, 2) **the binary codes distribute evenly on each bit**, and 3) **different bits are as independent as possible.**

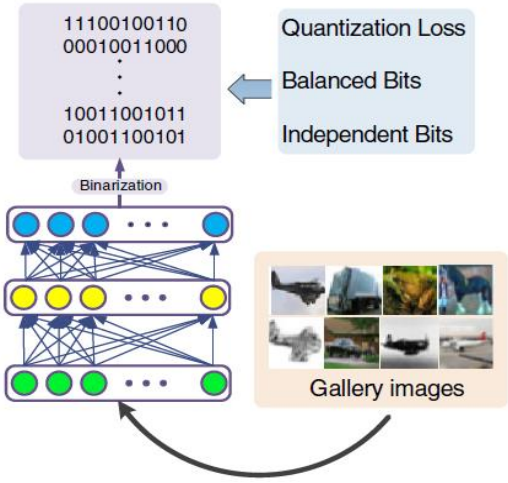
可学习的例句：

The basic idea of hashing-based approach is to construct a series of hash functions to map each visual object into a binary feature vector so that **visually similar samples are mapped into similar binary codes.**

实验设置：

For our DH and SDH methods, we trained our deep model with 3 layers by setting M=2, where the dimensions for these layers were empirically set as [60-30-**16**], [80-50-**32**], and [100-80-**64**] for the **16, 32 and 64** bits experiments, respectively.

We used the following three evaluation metrics to measure the performance of different methods: 1) **mean average precision (mAP):** which **computes the area under the precision-recall curve** and **evaluates the overall performance** of different hashing algorithms; 2) **precision at *N* samples**: which is **the percentage of true neighbors among top *N* retrieved samples**; and 3) **Hamming look-up result** when the **hamming radius is set as *r*:** which **measures the precision over all the points in the buckets that fall within a hamming radius of r=2 provided that a failed search would have zero precision**. We **repeated the experiments 10 times** and **took the average as the final result.**



$$\min_{\mathbf{W},\mathbf{c}} J = J_1 - \lambda_1 J_2 + \lambda_2 J_3 + \lambda_3 J_4$$
$$= \frac{1}{2}\|\mathbf{B} - \mathbf{H}^M\|_F^2 - \frac{\lambda_1}{2N}\mathrm{tr}((\mathbf{H}^M\mathbf{H}^M)^T)$$
$$+ \frac{\lambda_2}{2}\sum_{m=1}^{M}\|\mathbf{W}^m(\mathbf{W}^m)^T - \mathbf{I}\|_F^2$$
$$+ \frac{\lambda_3}{2}(\|\mathbf{W}^m\|_F^2 + \|\mathbf{c}^m\|_2^2) \quad (5)$$

The first term J_1 aims to minimize the quantization loss between the learned binary vectors and the original real-valued vectors. The second term J_2 aims to maximize the variance of learned binary vectors to ensure balanced bits. The third term J_3 enforces a relaxed orthogonality constraint on those projection matrices so that the independence of each transform is maximized. The last term J_4 are regularizers to control the scales of the parameters. λ_1, λ_2, and λ_3 are three parameters to balance the effect of different terms.

3.3. Supervised Deep Hashing

Since DH is an unsupervised learning approach, it is desirable to further improve its performance by using the label information of training samples. In this subsection, we propose a supervised deep hashing (SDH) method which extends DH into a supervised version to enhance the discriminative power of DH. For each pair of training samples $(\mathbf{x}_i, \mathbf{x}_j)$, we know whether they are from the same class or not. Hence, we can construct two sets $\mathcal{S}$ or $\mathcal{D}$ from the training set, which represents the positive samples pairs and the negative samples pairs in the training set, respectively. Then, we formulate the following optimization problem for our SDH method:

是否存在问题，W-B?

$$\arg\min_{\mathbf{W},\mathbf{c}} J = \frac{1}{2}\|\mathbf{B} - \mathbf{H}^M\|_F^2$$
$$- \frac{\lambda_1}{2}(\mathrm{tr}(\frac{1}{N}\mathbf{H}^M(\mathbf{H}^M)^T) + \boxed{\alpha\mathrm{tr}(\Sigma_B - \Sigma_W)}$$
$$+ \frac{\lambda_2}{2}\sum_{m=1}^{M}\|\mathbf{W}^m(\mathbf{W}^m)^T - \mathbf{I}\|_F^2$$
$$+ \frac{\lambda_3}{2}\sum_{m=1}^{M}(\|\mathbf{W}^m\|_F^2 + \|\mathbf{c}^m\|_2^2) \quad (12)$$

where

$$\Sigma_W = \frac{1}{N_S}\sum_{(\mathbf{x}_i,\mathbf{x}_j)\in\mathcal{S}}(\mathbf{h}_i^M - \mathbf{h}_j^M)(\mathbf{h}_i^M - \mathbf{h}_j^M)^T$$
$$= \frac{1}{N_S}\mathrm{tr}((\mathbf{H}_{s1}^M - \mathbf{H}_{s2}^M)(\mathbf{H}_{s1}^M - \mathbf{H}_{s2}^M)^T) \quad (13)$$
$$\Sigma_B = \frac{1}{N_D}\sum_{(\mathbf{x}_i,\mathbf{x}_j)\in\mathcal{D}}(\mathbf{h}_i^M - \mathbf{h}_j^M)(\mathbf{h}_i^M - \mathbf{h}_j^M)^T$$
$$= \frac{1}{N_D}\mathrm{tr}((\mathbf{H}_{d1}^M - \mathbf{H}_{d2}^M)(\mathbf{H}_{d1}^M - \mathbf{H}_{d2}^M)^T) \quad (14)$$

N_S and N_D are the number of neighbor and non-neighbor pairs[1], $\{\mathbf{H}_{s1}^m, \mathbf{H}_{s2}^m\}_{m=1}^M$ are the hidden representation of sample pairs in $\mathcal{S}$ and $\{\mathbf{H}_{d1}^m, \mathbf{H}_{d2}^m\}_{m=1}^M$ in $\mathcal{D}$. The objective of J_2 is to minimize the intra-class variations and maximize the inter-class variations, α is the parameter to balance these two parts in this term. The aims of J_1, J_3, and J_4 are the same as those of the DH method.

Similar to DH, we also use the stochastic gradient descent method to learn parameters $\{\mathbf{W}^m, \mathbf{c}^m\}_{m=1}^M$ in SDH.

[1]In our experiments, we randomly sampled 1000 positive and 1000 negative pairs from the training set to learn the SDH model.

2016

2016 IJCAI Feature Learning Based Deep Supervised Hashing with Pairwise Labels

Wu-Jun Li, Sheng Wang, Wang-Cheng Kang (with MATLAB code)

之前方法的缺点：

Most of these deep hashing methods are supervised whose supervised information is given **with triplet labels**. For another common application scenario **with pairwise labels, there have not existed methods for simultaneous feature learning and hash-code learning.**

To the best of our knowledge, **CNNH** is **the only one** which adopts deep neural network, which is actually a convolutional neural network (CNN) to perform feature learning for supervised hashing **with pairwise labels.**

In CNNH, the learned feature representation in the second stage **cannot give feedback** for learning better hash codes in the first stage. Hence, CNNH **cannot perform simultaneous feature learning and hash-code learning**, which still has limitations.

Most hashing methods **relax the binary codes to real-values in optimization** and **quantize the model outputs to produce binary codes.** However, **there is no guarantee that the optimal real-valued codes are still optimal after quantization.**

DPSH方法的优势：

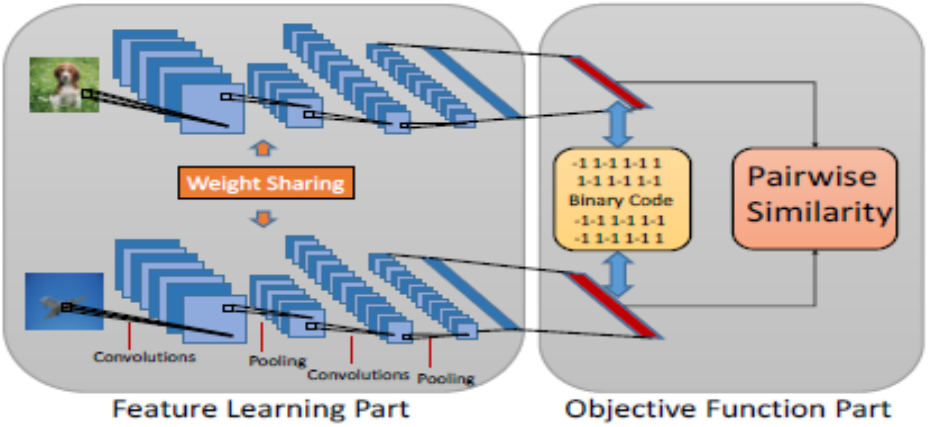
DPSH is an **end-to-end** learning framework which contains **three key components.** The **first component** is a deep neural network to **learn image representation** from pixels. The s**econd component** is a **hash function** to map the learned image representation to hash codes. And the **third component** is a **loss function** to measure the quality of hash codes guided by the pairwise labels. All the three components are **seamlessly integrated into the same deep architecture** to map the images from pixels to the pairwise labels in **an end-to-end way**. Hence, different components can **give feedback to each other** in DPSH, which results in learning better codes than other methods without end-to-end architecture.

To the best of our knowledge, **DPSH is the first method** which can perform **simultaneous feature learning and hash-code learning** for applications **with pairwise labels.**

$$\min_{\mathcal{B},\mathbf{W},\mathbf{v},\theta} \mathcal{J} = -\sum_{s_{ij}\in\mathcal{S}} \left(s_{ij}\Theta_{ij} - \log(1+e^{\Theta_{ij}})\right) + \eta \sum_{i=1}^{n} \|\mathbf{b}_i - (\mathbf{W}^T\phi(\mathbf{x}_i;\theta) + \mathbf{v})\|_2^2.$$

Table 1: Configuration of the feature learning part in DPSH.

Layer	Configuration
conv1	filter 64x11x11, stride 4x4, pad 0, LRN, pool 2x2
conv2	filter 256x5x5, stride 1x1, pad 2, LRN, pool 2x2
conv3	filter 256x3x3, stride 1x1, pad 1
conv4	filter 256x3x3, stride 1x1, pad 1
conv5	filter 256x3x3, stride 1x1, pad 1, pool 2x2
full6	4096
full7	4096



2016 CVPR Deep Supervised Hashing for Fast Image Retrieval

Haomiao Liu, Ruiping Wang, Shiguang Shan, Xilin Chen (with Caffe code)

之前方法的缺点：

However, the retrieval performance of most existing hashing methods **heavily depends on the features they use**, which are basically extracted in an **unsupervised manner,** thus **more suitable for dealing with the visual similarity search rather than the semantic similarity search.**

LSH methods use random projections to produce hashing bits. It has been proven theoretically that as the code length grows, **the Hamming distance between two binary codes asymptotically approaches their corresponding distance in the feature space.** However, **LSH methods usually require long codes to achieve satisfactory performance, which demands for large amount of memory.**

While the above methods **use linear projections as hashing functions,** they can **hardly deal with linearly inseparable data.**

They all use **hand-crafted features**, which **cannot capture the semantic information beneath the drastic appearance variations** in real-world data.

Nevertheless, there **still exist some shortcomings with the training objectives** of these methods that limit their practical retrieval performance, as will detailed in our experiments. In addition, **the non-linear activations they employ to approximate the quantization step operate at the cost of possibly slowing down the network training.**

DSH方法的优势：

CNNs can well capture the underlying semantic structure of images in spite of significant appearance variations.

In practice, we **generate image pairs online so that many more image pairs can be utilized in the training stage.**

To avoid optimizing the nondifferentiable loss function in Hamming space, the network **outputs are relaxed to real values,** while simultaneously **a regularizer is imposed** to **encourage** the real-valued **outputs** to **approach** the desired **discrete** values.

Our method **does not use saturating non-linearities**, e.g. tanh or sigmoid, to approximate the quantization step because **these nonlinearities are likely to slow down the training process.**

可学习的例句：

While the complex image appearance variations still pose a great challenge to reliable retrieval, in light of the recent progress of Convolutional Neural Networks (CNNs) in learning robust image representation on various vision tasks, this paper proposes a novel Deep Supervised Hashing (DSH) method to learn compact similarity-preserving binary code for the huge body of image data.

To this end, **a loss function is elaborately designed** to **maximize the discriminability of the output space** by encoding the supervised information from the input image pairs, and simultaneously imposing regularization on the real-valued outputs to approximate the desired discrete values.

For example, content based image retrieval retrieves images that are similar to a given query image, where **'similar' may refer to visually similar or semantically similar.**

Spectral Hashing minimizes the **weighted Hamming distance** of image pairs, where **the weights are defined to be the similarity metrics of image pairs.**

Iterative Quantization tries to **minimize the quantization error** on **projected image descriptors** so as to **alleviate the information loss caused by the discrepancy between the real-valued feature space and the binary Hamming space.**

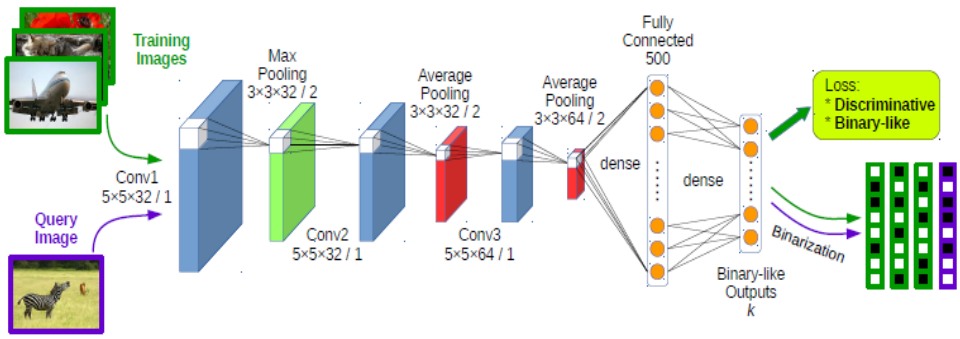


Figure 1. The network structure used in our method. The network consists of 3 convolution-pooling layers and 2 fully connected layers. The filters in convolution layers are of size 5×5 with stride 1 (32, 32, and 64 filters in the three convolution layers respectively), and pooling over 3×3 patches with stride 2. The first fully connected layer contains 500 nodes, and the second (output layer) has k (the code length) nodes. The loss function is designed to learn similarity-preserving binary-like codes by exploiting discriminability terms and a regularizer. Binary codes are obtained by quantizing the network outputs of images.

Specifically, for a pair of images $I_1, I_2 \in \Omega$ and the corresponding binary network outputs $\mathbf{b}_1, \mathbf{b}_2 \in \{+1, -1\}^k$, we define $y = 0$ if they are similar, and $y = 1$ otherwise. The loss with respect to the pair of images is defined as:

$$\begin{aligned} L(\mathbf{b}_1, \mathbf{b}_2, y) =& \frac{1}{2}(1-y)D_h(\mathbf{b}_1, \mathbf{b}_2) \\ &+ \frac{1}{2}y \max(m - D_h(\mathbf{b}_1, \mathbf{b}_2), 0) \\ & s.t.\ \mathbf{b}_j \in \{+1, -1\}^k,\ j \in \{1, 2\} \end{aligned} \tag{1}$$

where $D_h(\cdot, \cdot)$ denotes the Hamming distance between two binary vectors, and $m > 0$ is a margin threshold parameter. The first term punishes similar images mapped to different binary codes, and the second term punishes dissimilar images mapped to close binary codes when their Hamming distance falls below the margin threshold m. Here it is worth noting that to avoid collapsed solution, our loss function takes a contrastive loss form as [7] where only those dissimilar pairs having their distance within a radius are eligible to contribute to the loss function.

It would be preferable if one can directly optimize Eqn.(2), however it is infeasible because the binary constraints on $\mathbf{b}_{i,j}$ requires thresholding the network outputs (*e.g.* with signum function), and will make it intractable to train the network with back propagation algorithm. Some recent works [23, 16] propose to directly optimize the binary codes, however, due to the memory limitation, CNN models can only be trained with mini-batches, and the optimality of the produced binary codes is questionable when the batch size is very small compared to the whole training set.

On the other hand, if one totally ignores the binary constraints, it would result in suboptimal binary codes due to the discrepancy between the Euclidean space and the Hamming space. A commonly used relaxation scheme is to utilize *sigmoid* or *tanh* function to approximate the thresholding procedure. Nevertheless, working with such non-linear functions would inevitably slow down or even restrain the convergence of the network [12]. To overcome such limitation, in this work we propose to impose a regularizer on the real-valued network outputs to approach the desired discrete values (+1/-1). To be specific, we replace the Hamming distance in Eqn.(1) by Euclidean distance, and impose an additional regularizer to replace the binary constraints, then

$$\begin{aligned} \mathcal{L} = \sum_{i=1}^{N} L(\mathbf{b}_{i,1}, \mathbf{b}_{i,2}, y_i) \\ s.t.\ \mathbf{b}_{i,j} \in \{+1, -1\}^k,\ i \in \{1, ..., N\},\ j \in \{1, 2\} \end{aligned} \tag{2}$$

$$\begin{aligned} L_r(\mathbf{b}_1, \mathbf{b}_2, y) =& \frac{1}{2}(1-y)||\mathbf{b}_1 - \mathbf{b}_2||_2^2 \\ &+ \frac{1}{2}y \max(m - ||\mathbf{b}_1 - \mathbf{b}_2||_2^2, 0) \\ &+ \alpha(||\,|\mathbf{b}_1| - \mathbf{1}||_1 + ||\,|\mathbf{b}_2| - \mathbf{1}||_1) \end{aligned} \tag{3}$$

Here we use L2-norm to measure the distance between network outputs because **<u>the subgradients produced by lower-order norms treat the image pairs with different distances equally and thus make no use of the information involved in different distance magnitudes. (Why?)</u>** While higher-order norms are also feasible, more computations will be incurred accordingly at the same time. As for the regularizer, L1-norm is chosen rather than higher-order norms for **its much less computational cost,** which can favorably accelerate the training process.

这篇比较好的想法是将二值约束转化为求 b 的绝对值减 1 的 L1 范数最小，而不是使用饱和的非线性函数，文中只是说 L1 范数可以加快速度。

With this objective function, the network is trained using back-propagation algorithm with mini-batch gradient descent method. To do so, the gradients of Eqn.(4) w.r.t. $\mathbf{b}_{i,j}, \forall i, j$ need to be computed. Since the *max* operation and the absolute value operation in the objective function is non-differentiable at some certain points, we use subgradients instead, and define the subgradients to be 1 at such points. The subgradients of the first two terms of Eqn.(4) and that of the third term (*i.e.* the regularizer) are respectively written as:

$$\frac{\partial Term\ 1}{\partial \mathbf{b}_{i,j}} = (-1)^{j+1}(1-y_i)(\mathbf{b}_{i,1}-\mathbf{b}_{i,2})$$

$$\frac{\partial Term\ 2}{\partial \mathbf{b}_{i,j}} = \begin{cases} (-1)^{j} y_i(\mathbf{b}_{i,1}-\mathbf{b}_{i,2}), & ||\mathbf{b}_{i,1}-\mathbf{b}_{i,2}||_2^2 < m \\ \mathbf{0} & , otherwise \end{cases}$$

$$\frac{\partial Regularizer}{\partial \mathbf{b}_{i,j}} = \alpha\delta(\mathbf{b}_{i,j})$$

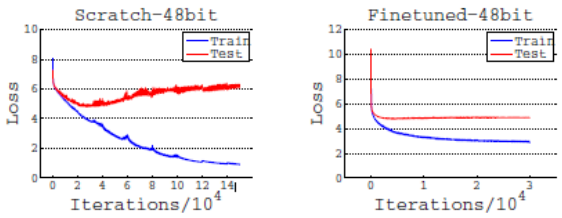
为什么是这个区间? (5)

where

为什么是0?

$$\delta(x) = \begin{cases} 1, & -1 \le x \le 0\ or\ x \ge 1 \\ -1, & otherwise \end{cases} \quad (6)$$

	Code Length	CIFAR-10	NUS-WIDE
Trained From Scratch	12	0.6157	0.5483
	24	0.6524	0.5543
	36	0.6433	0.5229
	48	0.6213	0.4896
Finetuned	24	0.6512	0.5513
	36	0.6607	0.5582
	48	0.6755	0.5621



(a) (b)

Figure 4. Comparison of (a) the model trained from scratch and (b) the finetuned model in terms of training/test loss, on the CIFAR-10 dataset.

精调比重新训练效果更好，可以避免过拟合

2016 AAAI Deep Hashing Network for Efficient Similarity Retrieval

Han Zhu, Mingsheng Long, Jianmin Wang, Yue Cao (with Caffe code)

之前方法的缺点：

For most existing supervised hashing methods for image retrieval, an image is first represented as a vector of **hand-crafted** or **machine-learned features, followed by another separate quantization step** that generates binary codes. However, **suboptimal hash coding** may be produced, because the **quantization error is not statistically minimized** and the **feature representation is not optimally compatible**

with the binary coding.

While **unsupervised methods** are more general and can be trained without semantic labels or relevances, **they are restricted by the semantic gap dilemma that high-level semantic description of an object often differs from low-level feature descriptors.**

However, a crucial disadvantage of these deep learning to hash methods is that **the quantization error is not statistically minimized** hence **the feature representation is not optimally compatible with binary hash coding.**

The **continuous relaxation**, i.e. solving **the discrete optimization of hash codes by more viable continuous optimization**, may give rise to **two important issues** widely ignored by previous learning to hash work: (1) **uncontrollable quantization error by binarizing continuous embeddings to binary codes**, and (2) **large approximation error by adopting ordinary distance between continuous embeddings as the surrogate of Hamming distance between binary codes.**

DHN方法的优势：

Supervised methods can **incorporate semantic labels or relevances to mitigate the semantic gap** and improve the hashing quality, i.e. **achieve accurate search with fewer bits of codes.**

We jointly **learn good image representation tailored to hash coding** and formally **control the quantization error.**

We simultaneously **control the quantization error** and **close the gap** between Hamming distance and its approximate distance for learning high-quality hash codes.

Devise a more principled pairwise cross entropy loss to link the pairwise Hamming distances with the pairwise similarity labels.

可学习的例句：

While image big data with large volume and high dimension **are pervasive in** search engines and social networks, it has attracted increasing attention to **enable approximate nearest neighbors (ANN) retrieval of images with both computation efficiency and search quality.**

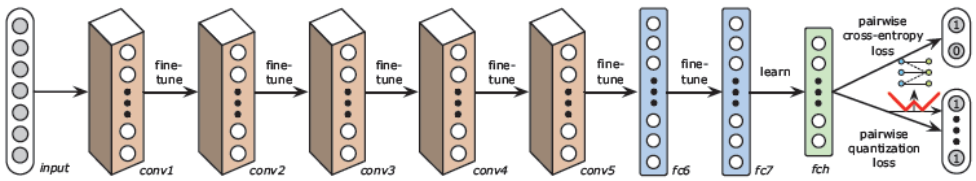


Figure 1: Deep Hashing Network (DHN) with a hash layer fch, a pairwise cross-entropy loss, and a pairwise quantization loss.

The DHN model constitutes four key components: (1) a subnetwork with multiple convolution-pooling layers to capture image representations; (2) a fully-connected hashing layer to generate compact binary hash codes; (3) a pairwise crossentropy loss layer for similarity-preserving learning; and (4) a pairwise quantization loss for controlling hashing quality.

使用双曲正切函数将网络输出压缩到区间[-1,1]. 在贝叶斯框架下同时保持成对相似度并控制量化误差.。使用内积来作为汉明距离的替代，以此量化成对相似度。使用了2013NIPS 中的结论，但没有相应引用。文中表述为:

$Dist_H$ (hi-hj)=1\2(K-<hi-hj>), 2013NIPS 中表述为: **<u, v> = (k – 2d$_h$ (u, v)), u, v \in {+-1}^k.**

$$\log p(\boldsymbol{H}|\mathcal{S}) \propto \log p(\mathcal{S}|\boldsymbol{H})\, p(\boldsymbol{H}) = \sum_{s_{ij}\in\mathcal{S}} \log p(s_{ij}|\boldsymbol{h}_i, \boldsymbol{h}_j)\, p(\boldsymbol{h}_i)\, p(\boldsymbol{h}_j), \quad (1)$$

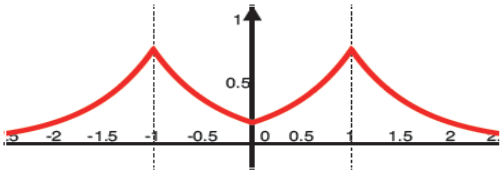
$$p(s_{ij}|\boldsymbol{h}_i, \boldsymbol{h}_j) = \begin{cases} \sigma(\langle \boldsymbol{z}_i^l, \boldsymbol{z}_j^l \rangle), & s_{ij} = 1 \\ 1 - \sigma(\langle \boldsymbol{z}_i^l, \boldsymbol{z}_j^l \rangle), & s_{ij} = 0 \end{cases} = \sigma(\langle \boldsymbol{z}_i^l, \boldsymbol{z}_j^l \rangle)^{s_{ij}} (1 - \sigma(\langle \boldsymbol{z}_i^l, \boldsymbol{z}_j^l \rangle))^{1-s_{ij}} \quad (2)$$

采用连续松弛来优化带有离散约束的目标，但是松弛会带了以下问题:

The continuous relaxation will give rise to two important issues widely ignored by previous learning to hash work: (1) **uncontrollable quantization error** by binarizing continuous embeddings to binary codes, and (2) **large approximation error** by adopting inner product between continuous embeddings as the surrogate of Hamming distance between binary codes.

We propose a novel **bimodal Laplacian prior** (unnormalized) for the continuous representations.

$$p(h_i) = \frac{1}{2\epsilon} \exp\left(-\frac{\left\| |h_i| - 1 \right\|_1}{\epsilon}\right), \quad (3)$$



参照了上面2016 CVPR的DSH，采用L1范数约束hi. 并且此双模态拉普拉斯先验灵感有可能产生于2016CVPR的实验结果图，如下:

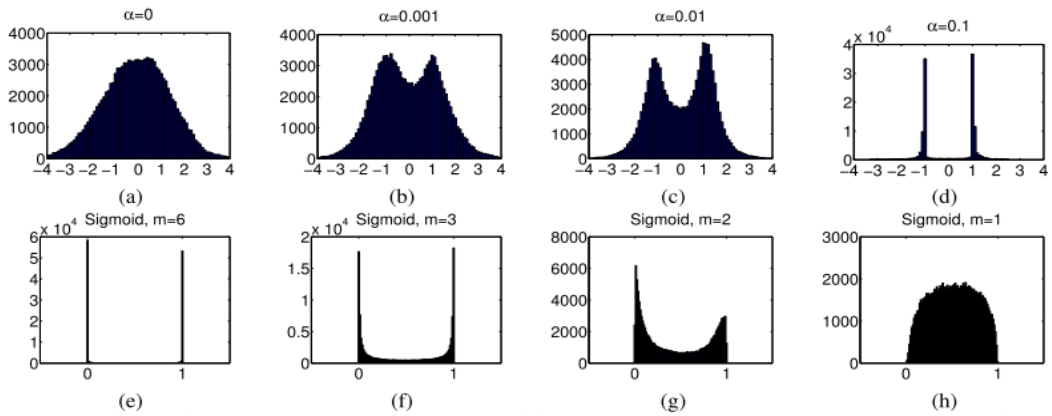


Figure 2. Distribution of network outputs on the test query set of CIFAR-10. (a)-(d) the models using our proposed regularizer under different settings of α, (e)-(h) the sigmoid relaxed models under different settings of m.

此外，本文目标函数结合了2014 ACM SIGIR 的LFH方法，只是解法有所不同。

$$\min_{\Theta} C = L + \lambda Q, \quad (4) \qquad Q = \sum_{s_{ij}\in\mathcal{S}} \left(\left\| |z_i^l| - 1 \right\|_1 + \left\| |z_j^l| - 1 \right\|_1 \right), \quad (6)$$

$$L = \sum_{s_{ij} \in \mathcal{S}} \left(\log \left(1 + \exp \left(\langle \boldsymbol{z}_i^l, \boldsymbol{z}_j^l \rangle \right) \right) - s_{ij} \langle \boldsymbol{z}_i^l, \boldsymbol{z}_j^l \rangle \right). \quad (5)$$

与上面2016 CVPR的DSH不同的是，为了方便L1范数的求导，此文采用光滑的替代:

where $\mathbf{1} \in \mathbb{R}^K$ is the vector of ones. As Q is a non-smooth function whose derivative is difficult to compute, we adopt a smooth surrogate (Hyvärinen, Hurri, and Hoyer 2009) of the absolute function $|x| \approx \log \cosh x$, which reduces (6) to

$$Q = \sum_{s_{ij} \in \mathcal{S}} \sum_{k=1}^{K} \left(\log \cosh \left(\left| z_{ik}^l \right| - 1 \right) + \log \cosh \left(\left| z_{jk}^l \right| - 1 \right) \right). \quad (7)$$

It is worth noting that, since we **have minimized the quantization error** in (4) during training, this final **binarization step will incur very small loss** of retrieval quality as validated empirically.

$$\begin{aligned} C_{ij} &\triangleq L_{ij} + \lambda Q_{ij} \\ &= \log \left(1 + \exp \left(\langle \boldsymbol{z}_i^l, \boldsymbol{z}_j^l \rangle \right) \right) - s_{ij} \langle \boldsymbol{z}_i^l, \boldsymbol{z}_j^l \rangle \\ &+ \lambda \sum_{k=1}^{K} \left(\log \cosh \left(\left| z_{ik}^l \right| - 1 \right) + \log \cosh \left(\left| z_{jk}^l \right| - 1 \right) \right). \end{aligned} \quad (8)$$

$$\begin{aligned} \frac{\partial C_i}{\partial \boldsymbol{W}_k^\ell} &= 2 \sum_{j: s_{ij} \in \mathcal{S}} \left(\frac{\partial L_{ij}}{\partial \boldsymbol{W}_k^\ell} + \lambda \frac{\partial Q_{ij}}{\partial \boldsymbol{W}_k^\ell} \right) \\ &= 2 \sum_{j: s_{ij} \in \mathcal{S}} \left(\frac{\partial L_{ij}}{\partial \hat{z}_{ik}^\ell} + \lambda \frac{\partial Q_{ij}}{\partial \hat{z}_{ik}^\ell} \right) \frac{\partial \hat{z}_{ik}^\ell}{\partial \boldsymbol{W}_k^\ell} \\ &= 2 \delta_{ik}^\ell \boldsymbol{z}_i^{\ell-1}, \end{aligned} \quad (9)$$

$$\begin{aligned} \delta_{ik}^l = &\sum_{j: s_{ij} \in \mathcal{S}} \left(\left[\sigma \left(\langle \boldsymbol{z}_i^l, \boldsymbol{z}_j^l \rangle \right) - s_{ij} \right] z_{jk}^l \right) \dot{a}^l \left(\hat{z}_{ik}^l \right) \\ &+ \lambda \sum_{j: s_{ij} \in \mathcal{S}} \tanh \left(\left| z_{ik}^l \right| - 1 \right) \operatorname{sgn} \left(z_{ik}^l \right) \dot{a}^l \left(\hat{z}_{ik}^l \right), \end{aligned} \quad (10)$$

$$\delta_{ik}^{\ell-1} = \left(\sum_{k'=1}^{u_\ell} \delta_{ik'}^\ell W_{k'k}^\ell \right) \dot{a}^{\ell-1} \left(\hat{z}_{ik}^{\ell-1} \right), \quad (11)$$

Theoretical Analysis

We elaborate the connection between our work and Iterative Quantization (ITQ) (Gong and Lazebnik 2011), the seminal work that considers the quantization error in an iterative Procrustean procedure. In ITQ, point-wise quantization error is

$$Q_{\text{ITQ}} = \|\boldsymbol{h}_i - \text{sgn}(\boldsymbol{h}_i)\|_2. \tag{12}$$

We only consider point-wise quantization error for brevity, while taking the summation for all training points (pairs) yields the overall quantization error similar to Equation (6).

Theorem 1 (Upper Bound). *The proposed quantization loss* (6) *is an upper bound of the ITQ quantization error* (12),

$$\|\boldsymbol{h}_i - \text{sgn}(\boldsymbol{h}_i)\|_2 \leq \||\boldsymbol{h}_i| - \mathbf{1}\|_1. \tag{13}$$

Proof. As $\boldsymbol{h}_i$ and $\text{sgn}(\boldsymbol{h}_i)$ have the same sign, it yields that

$$\begin{aligned} \|\boldsymbol{h}_i - \text{sgn}(\boldsymbol{h}_i)\|_2 &= \||\boldsymbol{h}_i| - |\text{sgn}(\boldsymbol{h}_i)|\|_2 \\ &= \||\boldsymbol{h}_i| - \mathbf{1}\|_2 \\ &\leq \||\boldsymbol{h}_i| - \mathbf{1}\|_1. \text{ (norm inequality)} \end{aligned} \tag{14}$$

□

Theorem 1 reveals that the proposed quantization loss in Equation (6) is a reasonable criterion for hash learning. The loss is easier to optimize in the back-propagation algorithm, since it does not involve a computation-intensive alternating optimization procedure as ITQ (Gong and Lazebnik 2011). Different from all existing methods, the proposed quantization loss is jointly optimized with the pairwise cross-entropy loss in a deep network, which yields better hashing schemes. An important advantage of the proposed loss is that the L_1-loss (6) may encourage sparsity, that is, more hash bits may be enforced to be $\{-1, 1\}$ compared with the L_2-loss (12), giving rise to more compact and discriminative hash codes.

实验结果(部分直接抄了其他文章的结果)**:**

To guarantee that our results directly comparable to most published results, **the results of LSH, BRE, ITQ, ITQ-CCA, KSH, MLH and SH** on both the NUS-WIDE and CIFAR-10 datasets **are directly reported from the latest work.**

These results validate that the proposed quantization loss (6) can effectively **reduce** the **binarization error** and lead to nearly lossless hash coding for high retrieval quality.

However, this loss does **not link well the pairwise Hamming distances** (taking values in (-∞, +∞) when using continuous relaxation) **to the pairwise similarity labels** (taking binary values {-1,1}). This is a **misspecified use of pairwise regression** for **pairwise classification problems**, which gives rise to **large similarity search errors**. The proposed pairwise cross-entropy loss (5) is derived rigorously from Bayesian learning framework (1) and is well-specified to the pairwise classification problem.

2016 IJCAI Deep Supervised Hashing with Triplet Labels

Xiaofang Wang, Yi Shi, Kris M. Kitani (with code)

之前方法的缺点：

Most traditional hashing methods **first represent images as off-the-shelf visual features and then produce hashing codes in a separate stage**. However, off-the-shelf visual features **may not be optimally compatible with the hash code learning procedure,** which may **result in sub-optimal hash codes.**

DPSH方法的优势：

Triplet labels的优点：

We argue that **triplet labels inherently contain richer information than pairwise labels**. **Each triplet label can be naturally decomposed into two pairwise labels.** Whereas, a triplet label can be constructed from two pairwise labels only when the same query image is used in a positive pairwise label and a negative pairwise label simultaneously. **A triplet label ensures that in the learned hash code space, the query image is close to the positive image and far from the negative image simultaneously.** However, a pairwise label can only ensure that one constraint is observed. **Triplet labels explicitly provide a notion of relative similarities between images while pairwise labels can only encode that implicitly.**

可学习的单词：

first represent images as off-the-shelf visual features.

off-the-shelf: 成品的；现成（用）的；流行的；畅销的；已准备好的；准军用的

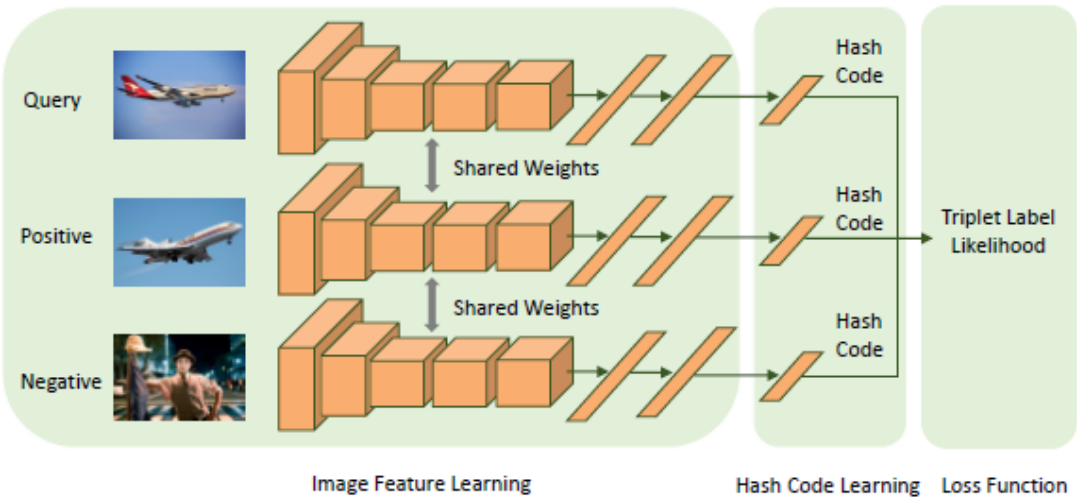


Fig. 1: Overview of the proposed end-to-end deep hashing method.

The proposed model has **three key components**: (1) **image feature learning component**: a deep neural network to learn visual features from images, (2) **hash code**

learning component: one fully connected layer to learn hash codes from image features and (3) **loss function component**: a loss function to measure how well the given triplet labels are satisfied by the learned hash codes by computing the likelihood of the given triplet labels. (使用CCN-F网络架构)

$$\Theta_{ij} = \frac{1}{2}\mathbf{b}_i^T\mathbf{b}_j \quad p(\mathcal{T} \mid \mathcal{B}) = \prod_{m=1}^{M} p((q_m, p_m, n_m) \mid \mathcal{B})$$

$$p((q_m, p_m, n_m) \mid \mathcal{B}) = \sigma(\Theta_{q_m p_m} - \Theta_{q_m n_m} - \alpha)$$

$$\mathrm{dist}_H(\mathbf{b}_i, \mathbf{b}_j) = \frac{1}{2}(L - 2\Theta_{ij})$$

$$\mathrm{dist}_H(\mathbf{b}_{q_m}, \mathbf{b}_{p_m}) - \mathrm{dist}_H(\mathbf{b}_{q_m}, \mathbf{b}_{n_m}) = -(\Theta_{q_m p_m} - \Theta_{q_m n_m})$$

$$\begin{aligned} L &= -\log p(\mathcal{T} \mid \mathcal{B}) \\ &= -\sum_{m=1}^{M} \log p((q_m, p_m, n_m) \mid \mathcal{B}) \end{aligned}$$

$$L = -\sum_{m=1}^{M} (\Theta_{q_m p_m} - \Theta_{q_m n_m} - \alpha - \log(1 + e^{\Theta_{q_m p_m} - \Theta_{q_m n_m} - \alpha}))$$

$$\Theta_{ij} = \frac{1}{2}\mathbf{u}_i^T\mathbf{u}_j$$

$$\begin{aligned} L = &-\sum_{m=1}^{M} (\Theta_{q_m p_m} - \Theta_{q_m n_m} - \alpha - \log(1 + e^{\Theta_{q_m p_m} - \Theta_{q_m n_m} - \alpha})) \\ &+ \lambda \sum_{n=1}^{N} ||\mathbf{b}_n - \mathbf{u}_n||_2^2 \end{aligned}$$

$$\begin{aligned} \frac{\partial L}{\partial \mathbf{u}_n} = &-\frac{1}{2} \sum_{m:(n,p_m,n_m)\in\mathcal{T}} (1 - \sigma(\Theta_{q_m p_m} - \Theta_{q_m n_m} - \alpha))(\mathbf{u}_{p_m} - \mathbf{u}_{n_m}) \\ &-\frac{1}{2} \sum_{m:(q_m,n,n_m)\in\mathcal{T}} (1 - \sigma(\Theta_{q_m p_m} - \Theta_{q_m n_m} - \alpha))\mathbf{u}_{q_m} \\ &+\frac{1}{2} \sum_{m:(q_m,p_m,n)\in\mathcal{T}} (1 - \sigma(\Theta_{q_m p_m} - \Theta_{q_m n_m} - \alpha))\mathbf{u}_{q_m} \\ &+ 2\lambda(\mathbf{u}_n - \mathbf{b}_n) \end{aligned}$$

The margin can also help speed up training our model as explained later in this Section and verified in our experiments.

2017

2017 ACM MM Deep Asymmetric Pairwise Hashing

Fumin Shen, Xin Gao, Li Liu, Yang Yang, Heng Tao Shen (without code)

之前方法的缺点：（目标函数参照2014的LFH，画图只对比了无监督的方法，部分对比数据直接引用原文，深度方法只做了MAP的实验，存在实验偷工减料的嫌疑，非对称在CIFAR-10上效果明显，其他数据集效果不明显，很有可能只是网络用的好，实验中没有说明测试样本的二值码产生于哪个网络，只说了训练集的）

In most existing pairwise-preserving supervised hashing methods, each input image is firstly represented by a vector of traditional **hand-crafted visual descriptors**, followed by **separate projection and quantization steps** to encode image vector into binary codes. **The hand-crafted features might not achieve satisfactory performance with the binary code learning procedure.**

However, some deep learning to hash methods is **symmetric hashing scheme** which similarity between two objects is approximated by the Hamming distance between **the outputs of the same hash function.**

The symmetric discrete constraint might be difficult to optimize and not necessary to learn binary codes of training data.

DAPH方法的优势：

DAPH is the first **deep asymmetric hashing** method which integrates the feature learning and asymmetric hash functions learning into the end-to-end deep learning framework.

DAPH has two advantages: (1) We propose to **use two deep neural networks** to generate asymmetric hashing functions and learn efficient image representation simultaneously. (2) We **adopt the independence and balance properties** to the binary codes so as to generate high-quality binary code.

The asymmetric discrete matrices can preserve more similarity (label) information, which usually means better retrieval accuracy.

Deep neural networks learn the image representation and hash codes **in one stage** so that representation learning and hash learning are **tightly coupled** to benefit each other.

可学习的例句：

These compact codes can **substantially reduce the storage overhead** and also **speed up image search significantly.**

Unsupervised hashing methods attempt to **integrate the data properties**, such as **data distributions and manifold structures** to design compact hash codes with improved accuracy.

Supervised methods can **incorporate semantic labels or relevance** to **mitigate the semantic gap** and has demonstrated superior image search accuracy compared with unsupervised hashing algorithms.

Pairwise similarity is widely adopted to make the distances of similarity pairs in

original space and Hamming space **as consistent as possible**, i.e., semantically similar images should have similar binary codes.

The features learned by deep neural networks can **well capture the underlying semantic structure of images in spite of significant appearance variations.**

The **substantial superior performance** of DAPH **verifies the benefit of joint learning feature representations and hash codes rather than the traditional cascaded scheme** (i.e. feature extraction followed by hashing).

The **underlying reason** why our method **exceeds DHN method by a large margin** may be that by using two deep networks to **construct distinct asymmetric hash functions**, our methods can **capture more information** and has higher learning capability and is able to **exploit more semantic information than symmetric hashing setting. (**另一方面原因就是网络架构好罢了，用的残差网络**)**

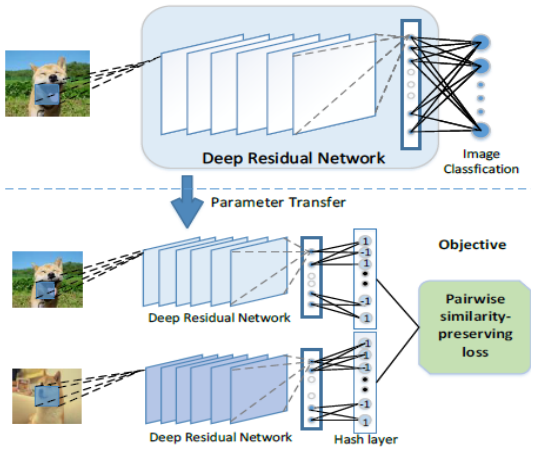


Figure 1: Overview of the proposed DAPH method The training procedure includes two stages: the pre-training with the deep residual network (top) and the deep asymmetric hash model training (bottom).

The core idea is that **two deep convolutional models are jointly trained such that their output codes for a pair of images can well reveal the similarity indicated by their semantic labels.** A pairwise loss is elaborately designed to **preserve the pairwise similarities** between images **as well as incorporating the independence and balance hash code learning criteria.**

We use two deep neural network to **construct asymmetric hash functions** to learn directly from images, which provides much information than hand-crafted features.

$$L = \sum_{s_{ij} \in S} (log(1 + e^{\Theta_{ij}}) - s_{ij}\Theta_{ij}) \tag{3}$$

$$p(s_{ij}|b_i, h_j) = \begin{cases} \sigma(\Theta_{ij}) & s_{ij} = 1 \\ 1 - \sigma(\Theta_{ij}) & s_{ij} = 0 \end{cases} \tag{4}$$

Furthermore, a good hashing method should produce binary codes with the properties: (1) **independence**, i.e., different bits in the binary codes are independent to each other; (2) **balance**, i.e. each bit hash a 50% chance of being 1 or -1.

$$
\begin{aligned}
\min_{B,H} \quad L \quad &= \sum_{s_{ij}\in S}\left(\log\left(1+e^{\Theta_{ij}}\right)-s_{ij}\Theta_{ij}\right) \\
&+\frac{\lambda}{2}\left(\left\|\frac{1}{N}BB^T-I\right\|^2+\left\|\frac{1}{N}HH^T-I\right\|^2\right) \\
&+\frac{\beta}{2}\left(\|B1_{N\times 1}\|^2+\|H1_{N\times 1}\|^2\right) \qquad (5) \\
\text{s.t.} \quad & B,H\in\{-1,1\}^{k\times N} \qquad (6)
\end{aligned}
$$

To be specific, we **replace the binary constraint** b_i, h_j in (5) **by the real-valued network outputs** u_i and z_j respectively.

$$
\begin{aligned}
\min_{B,H,U,Z} \quad L &= \sum_{s_{ij}\in S}\left(\log\left(1+e^{\Theta_{ij}}\right)-s_{ij}\Theta_{ij}\right) \\
&+\frac{\alpha}{2}\left(\sum_{i=1}^{n}\|u_i-b_i\|^2+\sum_{j=1}^{n}\|z_j-h_j\|^2\right) \\
&+\frac{\lambda}{2}\left(\left\|\frac{1}{N}UU^T-I\right\|^2+\left\|\frac{1}{N}ZZ^T-I\right\|^2\right) \\
&+\frac{\beta}{2}\left(\|U1_{N\times 1}\|^2+\|Z1_{N\times 1}\|^2\right) \qquad (7) \\
s.t. \quad & B,H\in\{-1,1\}^{k\times N}. \qquad (8)
\end{aligned}
$$

In our model, **the binary code matrix for the training data X generated by the two asymmetric hash functions should be as close as possible.**

$$
\begin{aligned}
\min_{B,H,U,Z} \quad L &= \sum_{s_{ij}\in S}\left(\log\left(1+e^{\Theta_{ij}}\right)-s_{ij}\Theta_{ij}\right) \\
&+\frac{\alpha}{2}\left(\|U-B\|^2+\|Z-H\|^2\right)+\frac{\gamma}{2}\left(\|B-H\|^2\right) \\
&+\frac{\lambda}{2}\left(\left\|\frac{1}{N}UU^T-I\right\|^2+\left\|\frac{1}{N}ZZ^T-I\right\|^2\right) \\
&+\frac{\beta}{2}\left(\|U1_{N\times 1}\|^2+\|Z1_{N\times 1}\|^2\right) \qquad (9) \\
\text{s.t.} \quad & B,H\in\{-1,1\}^{k\times N}. \qquad (10)
\end{aligned}
$$

As well as image search, **binary hashing** has recently been applied to boost many other tasks, such as **classification** [26], **clustering** [34, 38], **recommendation systems** [41] and **action recognition** [23].

2017 NIPS Deep Supervised Discrete Hashing

Qi Li, Zhenan Sun, Ran He, Tieniu Tan (without code)

之前方法的缺点：

However, there are some limitations of previous deep hashing methods (e.g., **the semantic information is not fully exploited**).

Recent works try to divide the whole learning process into two streams under the

multitask learning framework [11, 21, 22]. The hash stream is used to learn the hash function, while the classification stream is utilized to mine the semantic information. **Although the two stream framework can improve the retrieval performance**, **the classification stream is only employed to learn the image representations**, **which does not have a direct impact on the hash function.**

Most of the previous works make **use of the label information under a two stream multi-task learning framework**. The **classification stream is used to measure the classification error**, while **the hash stream is employed to learn the hash function.**

DSDH方法的优势：

We develop a deep supervised discrete hashing algorithm based on the assumption that **the learned binary codes should be ideal for classification. Both the pairwise label information and the classification information are used** to learn the hash codes within one stream framework. We **constrain the outputs of the last layer to be binary codes directly,** which is rarely investigated in deep hashing algorithm.

The last layer of our method is constrained to output the binary codes directly. **The binary codes are learned to preserve the similarity relationship and keep the label consistent simultaneously**. To the best of our knowledge, **this is the firs deep hashing method that uses both pairwise label information and classification information to learn the hash codes under one stream framework.**

One basic assumption of our algorithm is that the learned binary codes should be ideal for classification. In order to take advantage of the label information directly, we expect the learned binary codes to be optimal for the jointly learned linear classifier.

We constrain the outputs of the last layer to be binary codes directly.

这篇文章口口声声说在最后一层直接是二值码，但是优化过程中却又换成了连续值，文章比较水，还能发NIPS。

$$F = -\sum_{s_{ij}\in S}\left(s_{ij}\Phi_{ij} - \log\left(1+e^{\Phi_{ij}}\right)\right) + \mu\sum_{i=1}^{N}\left\|y_i - W^T b_i\right\|_2^2 + \nu\left\|W\right\|_F^2, \tag{7}$$

The hypothesis for Equation 7 is that **the learned binary codes should make the pairwise label likelihood as large as possible, and should be optimal for the jointly learned linear classifier.**

$$
\begin{aligned}
&F = -\sum_{s_{ij}\in S}\left(s_{ij}\Psi_{ij} - \log\left(1+e^{\Psi_{ij}}\right)\right) + \mu\sum_{i=1}^{N}\left\|y_i - W^T b_i\right\|_2^2 + \nu\left\|W\right\|_F^2, \\
&s.t.\ \ b_i = \mathrm{sgn}(h_i),\ \ h_i \in \mathbb{R}^{K\times 1},\ \ (i=1,...,N),
\end{aligned} \tag{8}
$$

where $\Psi_{ij} = \frac{1}{2}{h_i}^T h_j$. h_i $(i = 1, ..., N)$ can be seen as the output of the last fully connected layer, which is represented as:

$$
h_i = M^T\Theta\left(x_i;\theta\right) + n, \tag{9}
$$

where θ denotes the parameters of the previous layers before the last fully connected layer, $M \in \mathbb{R}^{4096\times K}$ represents the weight matrix, $n \in \mathbb{R}^{K\times 1}$ is the bias term.

According to the Lagrange multipliers method, Equation 8 can be reformulated as:

$$
\begin{aligned}
&F = -\sum_{s_{ij}\in S}\left(s_{ij}\Psi_{ij} - \log\left(1+e^{\Psi_{ij}}\right)\right) \\
&+\mu\sum_{i=1}^{N}\left\|y_i - W^T b_i\right\|_2^2 + \nu\left\|W\right\|_F^2 + \eta\sum_{i=1}^{N}\left\|b_i - \mathrm{sgn}\left(h_i\right)\right\|_2^2, \\
&s.t.\quad b_i \in \{-1,1\}^K,\ \ (i=1,...,N),
\end{aligned} \tag{10}
$$

where η is the Lagrange Multiplier. Equation 10 can be further relaxed as:

$$
\begin{aligned}
&F = -\sum_{s_{ij}\in S}\left(s_{ij}\Psi_{ij} - \log\left(1+e^{\Psi_{ij}}\right)\right) \\
&+\mu\sum_{i=1}^{N}\left\|y_i - W^T b_i\right\|_2^2 + \nu\left\|W\right\|_F^2 + \eta\sum_{i=1}^{N}\left\|b_i - h_i\right\|_2^2, \\
&s.t.\quad b_i \in \{-1,1\}^K,\ \ (i=1,...,N).
\end{aligned} \tag{11}
$$

The benefit of introducing an auxiliary variable is that we can decompose Equation 11 into two sub optimization problems, which can be iteratively solved by using the alternating minimization method.

First, when fixing b_i, W, we have:

$$
\frac{\partial F}{\partial h_i} = -\frac{1}{2}\sum_{j:s_{ij}\in S}\left(s_{ij} - \frac{e^{\Psi_{ij}}}{1+e^{\Psi_{ij}}}\right)h_j - \frac{1}{2}\sum_{j:s_{ji}\in S}\left(s_{ji} - \frac{e^{\Psi_{ji}}}{1+e^{\Psi_{ji}}}\right)h_j - 2\eta\left(b_i - h_i\right) \tag{12}
$$

Then we update parameters M, n and Θ as follows:

$$
\frac{\partial F}{\partial M} = \Theta\left(x_i;\theta\right)\left(\frac{\partial F}{\partial h_i}\right)^T,\ \frac{\partial F}{\partial n} = \frac{\partial F}{\partial h_i},\ \frac{\partial F}{\partial \Theta(x_i;\theta)} = M\frac{\partial F}{\partial h_i}. \tag{13}
$$

The gradient will propagate to previous layers by Back Propagation (BP) algorithm.

Second, when fixing M, n, Θ and b_i, we solve W as:

$$
F = \mu\sum_{i=1}^{N}\left\|y_i - W^T b_i\right\|_2^2 + \nu\left\|W\right\|_F^2. \tag{14}
$$

Equation 14 is a least squares problem, which has a closed form solution:

$$
W = \left(BB^T + \frac{\nu}{\mu}I\right)^{-1}B^T Y, \tag{15}
$$

where $B = \{b_i\}_{i=1}^N \in \{-1,1\}^{K\times N}, Y = \{y_i\}_{i=1}^N \in \mathbb{R}^{C\times N}$.

Finally, when fixing M, n, Θ and W, Equation 11 becomes:

$$
\begin{aligned}
&F = \mu\sum_{i=1}^{N}\left\|y_i - W^T b_i\right\|_2^2 + \eta\sum_{i=1}^{N}\left\|b_i - h_i\right\|_2^2, \\
&s.t.\quad b_i \in \{-1,1\}^K, (i=1,...,N).
\end{aligned} \tag{16}
$$

In this paper, we use the discrete cyclic coordinate descend method to iteratively solve B row by row:

$$
\min_B\ \left\|W^T B\right\|^2 - 2\,\mathrm{Tr}\left(P\right),\quad s.t.\ \ B \in \{-1,1\}^{K\times N}, \tag{17}
$$

where $P = WY + \frac{\eta}{\mu}H$. Let x^T be the kth $(k = 1, ..., K)$ row of B, B_1 be the matrix of B excluding x^T, p^T be the kth column of matrix P, P_1 be the matrix of P excluding p, w^T be the kth column of matrix W, W_1 be the matrix of W excluding w, then we can derive:

$$
x = \mathrm{sgn}\left(p - B_1^T W_1 w\right). \tag{18}
$$

It is easy to see that each bit of the hash codes is computed based on the pre-learned $K-1$ bits B_1. We iteratively update each bit until the algorithm converges.

2017 IJCAI Semi-Supervised Deep Hashing with a Bipartite Graph

Xinyu Yan, Lijun Zhang, Wu-Jun Li (without code)

之前方法的缺点：（个人感觉此文实验有些简单，就两个数据库三个实验，为什么不对比三种不同的图的效果？没有说明图嵌入的损失函数用的是哪个？实验也没有分析BGDH-T和BGDH-I的差别和产生差别的原因）

However, most of existing deep hashing methods are designed for the supervised scenario and **require a large number of labeled data.**

However, **labeling** large scale image dataset is **inefficient and time-consuming.**

SSDH is fundamentally built upon graph-based semi-supervised learning and the loss function contains a graph regularization term which involves both the labeled and unlabeled data. In theory, **SSDH needs to construct a nearest neighbor graph of all the data. Unfortunately, this step takes O(n^2) time, where n is the number of instances, and thus intractable for large scale data.**

BGDH is a general learning framework in the sense that **any loss function of hashing and any type of graph can be incorporated.**

BGDH方法的优势：

Deep Hashing with a Bipartite Graph (BGDH), which **performs graph embedding, feature learning and hash code learning in a unified framework.**

可学习的例句：

Finally, **we emphasize that although both BGDH and SSDH are semi-supervised deep hashing methods, the proposed BGDH differs from SSDH in the following two aspects:**

a. While **SSDH is built upon graph regularization, our BGDH relies on graph embedding.**

b. SSDH uses graphs to exploit the unlabeled data, in contract **BGDH makes use of bipartite graphs, which can be constructed more efficiently since building an anchor graph only costs O(n) time.**

As can be seen, **there is a wide range of lambda that BGDH performs well**. Thus, **to a large extent, BGDH is insensitive to lambda and the parameter selection is not a crucial problem in our algorithm.** Additionally, by comparing the MAP of lambda = 0 and lambda = 1, we **verify the importance of graph embedding.**

$$\mathcal{L}_g = \frac{1}{t}\sum_{j=1}^{t} \ell(\mathbf{e}_{i_j}^{\top}\mathbf{w}_{c_j}, \gamma_j) \qquad (2)$$

$$\mathbf{u}_i = \mathbf{M}^T[\phi(\mathbf{x}_i;\theta_x);\psi(\mathbf{e}_i;\theta_e)] + \mathbf{v} \qquad (3)$$

$$\mathcal{L}_s = -\sum_{s_{ij}\in\mathcal{S}} (s_{ij}\Theta_{ij} - \log(1+e^{\Theta_{ij}})) + \eta\sum_{i=1}^{l}\|\mathbf{b}_i - \mathbf{u}_i\|_2^2 \qquad (4)$$

$$\mathcal{L}_s = -\sum_{s_{ij}\in\mathcal{S}} \left(s_{ij}\Theta_{ij} - \log(1+e^{\Theta_{ij}})\right) + \eta \sum_{i=1}^{l} \left\|\mathbf{b}_i - (\mathbf{M}^T[\phi(\mathbf{x}_i;\theta_x);\psi(\mathbf{e}_i;\theta_e)] + \mathbf{v})\right\|_2^2 . \tag{5}$$

$$\mathcal{L}_s + \lambda\mathcal{L}_g = -\sum_{s_{ij}\in\mathcal{S}} \left(s_{ij}\Theta_{ij} - \log(1+e^{\Theta_{ij}})\right) + \eta \sum_{i=1}^{l} \left\|\mathbf{b}_i - (\mathbf{M}^T[\phi(\mathbf{x}_i;\theta_x);\psi(\mathbf{e}_i;\theta_e)] + \mathbf{v})\right\|_2^2 + \frac{\lambda}{t}\sum_{j=1}^{t} \ell(\mathbf{e}_{i_j}^\top \mathbf{w}_{c_j}, \gamma_j) \tag{6}$$

$$\mathbf{b}_i = \mathrm{sgn}(\mathbf{u}_i) = \mathrm{sgn}\left(\mathbf{M}^T[\phi(\mathbf{x}_i;\theta_x);\psi(\mathbf{e}_i;\theta_e)] + \mathbf{v}\right) \tag{9}$$

$$\frac{\partial\mathcal{L}_s}{\partial\mathbf{u}_i} = \frac{1}{2}\sum_{j:s_{ij}\in\mathcal{S}} (a_{ij} - s_{ij})\mathbf{u}_j + \frac{1}{2}\sum_{j:s_{ji}\in\mathcal{S}} (a_{ji} - s_{ji})\mathbf{u}_j + 2\eta(\mathbf{u}_i - \mathbf{b}_i) \tag{10}$$

$$\frac{\partial\mathcal{L}_s}{\partial\mathbf{M}} = [\phi(\mathbf{x}_i;\theta_x);\psi(\mathbf{e}_i;\theta_e)]\left(\frac{\partial\mathcal{L}_s}{\partial\mathbf{u}_i}\right)^T, \tag{11}$$

$$\frac{\partial\mathcal{L}_s}{\partial\mathbf{v}} = \frac{\partial\mathcal{L}_s}{\partial\mathbf{u}_i}, \tag{12}$$

$$\frac{\partial\mathcal{L}_s}{\partial\phi(\mathbf{x}_i;\theta_x)} = \frac{\partial\mathcal{L}_s}{\partial\psi(\mathbf{e}_i;\theta_e)} = \mathbf{M}\frac{\partial\mathcal{L}_s}{\partial\mathbf{u}_i}. \tag{13}$$

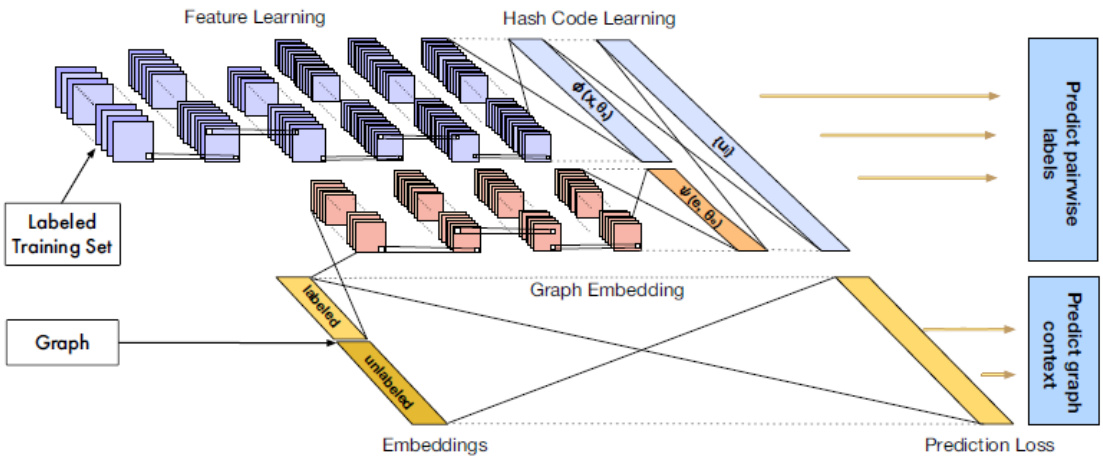


Figure 1: The end-to-end deep learning architecture of BGDH.

2017 ICCV HashNet: Deep Learning to Hash by Continuation

Zhangjie Cao, Mingsheng Long, Jianmin Wang, Philip S. Yu (with Pytorch code)

之前方法的缺点：（个人感觉此文写的比较好，实验也很充分，证明收敛性部分

没有看，看上去很复杂的样子）

Subject to the ill-posed gradient difficulty in the optimization with sign activations, **existing deep learning to hash methods need to firs learn continuous representations and then generate binary hash codes in a separated binarization step, which suffer from substantial loss of retrieval quality.**

However, **a key disadvantage of these deep learning to hash methods is that they need to first learn continuous deep representations, which are binarized into hash codes in a separated post-step of sign thresholding.**

By continuous relaxation, i.e. solving the discrete optimization of hash codes with continuous optimization, **all these methods essentially solve an optimization problem that deviates significantly from the hashing objective as they cannot learn exactly binary hash codes in their optimization procedure.**

Different from HashNet, **previous deep-hashing methods need to first learn continuous embeddings, which are binarized in a separated step using the sign function. This will result in substantial quantization errors and significant losses of retrieval quality.**

HashNet方法的优势：

A novel deep architecture for deep learning to hash by continuation method with convergence guarantees, which **learns exactly binary hash codes from imbalanced similarity data.**

This work presents HashNet, a new architecture for **deep learning to hash by continuation with convergence guaran tees,** which **addresses the ill-posed gradient and data imbalance problems in an end-to-end framework** of deep feature learning and binary hash encoding.

This work is the **first effort to learn sign-activated deep networks by continuation method, which can directly optimize L(h) for deep learning to hash.**

We attack the ill-posed gradient problem of the non-smooth activation function h = sgn (z) by continuation, which **starts with a smoothed activation function y = tanh (\beta x) and becomes more non-smooth by increasing \beta as the training proceeds, until eventually goes back to the original, difficult to optimize, sign activation function.**

It is noteworthy that **our work is the first attempt that extends the WML estimation from pointwise scenario to pairwise scenario.**

In contrast, **the proposed weighted maximum likelihood estimation (1) is a principled solution to tackling the data imbalance problem by weighting the training pairs according to the importance of misclassifying that pair.**

The proposed continuation method is a principled solution to deep learning to hash with sgn() as activation function, which learn lossless binary hash codes for accurate retrieval.

可学习的例句：

There are two key challenges to enabling deep learning to hash truly end-to-end First, converting deep representations, which are continuous in nature, to exactly binary hash codes, **we need to adopt the sign function as h = sgn (z) activation function**

when generating binary hash codes using similarity-preserving learning in deep neural networks. However, **the gradient of the sign function is zero for all nonzero inputs, which will make standard back-propagation infeasible.** This is known as the **ill-posed gradient problem**, which is the key difficulty in training deep neural networks via back-propagation. Second, **the similarity information is usually very sparse in real retrieval systems**, i.e., the number of similar pairs is much smaller than the number of dissimilar pairs. This will result in the **data imbalance problem, making similarity-preserving learning ineffective.**

Unfortunately, as **the sign function is non-smooth and non-convex, its gradient is zero for all nonzero inputs, and is ill-defined at zero,** which makes the standard back-propagation infeasible for training deep networks. **This is known as the vanishing gradient problem, which has been a key difficult in training deep neural networks via back-propagation.**

When using longer codes, the Hamming space will become sparse and few data points fall within the Hamming ball with radius 2. This is why most hashing methods achieve best accuracy with moderate code lengths.

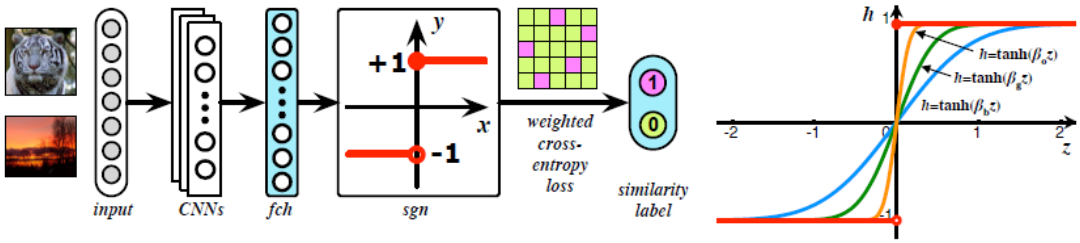


Figure 1. (left) The proposed HashNet for deep learning to hash by continuation, which is comprised of four key components: (1) Standard convolutional neural network (CNN), e.g. AlexNet and ResNet, for learning deep image representations, (2) a fully-connected hash layer (fch) for transforming the deep representation into K-dimensional representation, (3) a sign activation function (sgn) for binarizing the K-dimensional representation into K-bit binary hash code, and (4) a novel weighted cross-entropy loss for similarity-preserving learning from sparse data. (right) Plot of smoothed responses of the sign function $h = \mathrm{sgn}(z)$: Red is the sign function, and blue, green and orange show functions $h = \tanh(\beta z)$ with bandwidths $\beta_b < \beta_g < \beta_o$. The key property is $\lim_{\beta\to\infty} \tanh(\beta z) = \mathrm{sgn}(z)$. *Best viewed in color.*

$$\log P(\mathcal{S}|\boldsymbol{H}) = \sum_{s_{ij}\in\mathcal{S}} w_{ij} \log P(s_{ij}|\boldsymbol{h}_i, \boldsymbol{h}_j), \qquad (1)$$

$$w_{ij} = c_{ij} \cdot \begin{cases} |\mathcal{S}| \,/\, |\mathcal{S}_1|, & s_{ij} = 1 \\ |\mathcal{S}| \,/\, |\mathcal{S}_0|, & s_{ij} = 0 \end{cases} \qquad (2)$$

$$\min_{\Theta} \sum_{s_{ij}\in\mathcal{S}} w_{ij} \left(\log\left(1 + \exp\left(\alpha \langle \boldsymbol{h}_i, \boldsymbol{h}_j \rangle\right)\right) - \alpha s_{ij} \langle \boldsymbol{h}_i, \boldsymbol{h}_j \rangle\right), \qquad (4)$$

$$\lim_{\beta\to\infty} \tanh(\beta z) = \mathrm{sgn}(z), \qquad (6)$$

Theorem 1. *The HashNet loss L will not change across stages t and $t+1$ with bandwidths switched from β_t to β_{t+1}.*

Theorem 2. *Loss L decreases when optimizing loss $J(g)$ by the stochastic gradient descent (SGD) within each stage.*

2017 CVPR Deep Sketch Hashing: Fast Free-hand Sketch-Based Image Retrieval

Li Liu, Fumin Shen, Yuming Shen, Xianglong Liu, Ling Shao (with code)

2018

2018 CVPR Deep Cauchy Hashing for Hamming Space Retrieval

Yue Cao, Mingsheng Long, Bin Liu, Jianmin Wang (with Pytorch code)

之前方法的缺点：（文章没有说明目标函数的优化过程）

However, **subject to the weak capability of concentrating relevant images to be within a small Hamming ball due to mis-specified loss functions**, existing deep hashing methods may **underperform for Hamming space retrieval.**

However, most of the **existing methods are tailored to data compression instead of candidates pruning, i.e. they are designed to maximize retrieval performance based on linear scan of hash codes.** As **linear scan is still costly even using hash codes,** we are somewhat deviating from our original intention with hashing, that is, to maximize speedup under acceptable accuracy.

In Hamming space retrieval, **we return data points within a given Hamming radius to each query, by hash table lookups instead of linear scan.** Unfortunately,

existing hashing methods generally lack the capability of concentrating relevant images to be within a small Hamming ball due to the mis-specified loss functions, thus they may underperform for Hamming space retrieval.

However, **previous deep hashing methods perform unsatisfactorily for Hamming space retrieval, with early pruning to discard irrelevant data points out of Hamming Radius 2. Their loss functions penalize little when two similar data points have large Hamming distance. Thus they cannot concentrate relevant data points to be within Hamming radius 2.**

This implies that **previous deep hashing methods cannot pull the Hamming distance between the hash codes of similar data points to be smaller than 2, because the probabilities for different Hamming distances smaller than K=2 are not discriminative enough.**

DCH方法的优势：

We propose a pairwise cross-entropy loss based on Cauchy distribution, **which penalizes significantly on similar image pairs with Hamming distance larger than the given Hamming radius threshold.**

We further propose a quantization loss based on the Cauchy distribution, **which enables learning nearly lossless hash codes.**

We propose **a novel cross-entropy loss for similarity-preserving learning and a novel quantization loss for controlling hashing quality, both based on the Cauchy distribution.** To our knowledge, this work is **the first endeavor towards deep hashing for Hamming space retrieval.**

Note that, **a well-specified loss function for Hamming space retrieval should penalize significantly for similar points with Hamming distance greater than 2, while prior methods are misspecified for this goal.**

可学习的例句：

With compact hash codes, Hamming space retrieval **enables the most efficient constant-time search that returns data points within a given Hamming radius to each query,** by hash table lookups instead of linear scan.

Parallel to the traditional indexing methods for candidates pruning, another advantageous solution is **hashing methods for data compression**, which transform high-dimensional media data into compact binary codes and generate similar binary codes for similar data items.

They prove **it crucial to jointly learn similarity-preserving representations and control quantization error of converting continuous representations to binary codes.**

Due to the potentially best efficiency, the search based on pruning followed by re-ranking is widely-deployed in large-scale online retrieval systems such as search engines.

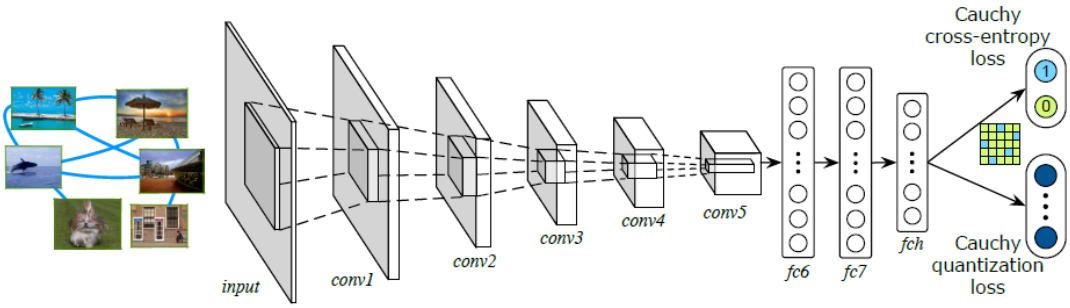


Figure 1. The architecture of the proposed Deep Cauchy Hashing (DCH), which is comprised of four key components: (**1**) a convolutional network (CNN) for learning deep representation of each image x_i, (**2**) a fully-connected hash layer (*fch*) for transforming the deep representation into K-bit hash code $h_i \in \{1, -1\}^K$, (**3**) a novel Cauchy cross-entropy loss for similarity-preserving learning in the Hamming space, and (**4**) a novel Cauchy quantization loss for controlling both the binarization error and the hash code quality. *Best viewed in color.*

$$\begin{aligned}\log P\left(\boldsymbol{H}|\mathcal{S}\right) &\propto \log P\left(\mathcal{S}|\boldsymbol{H}\right) P\left(\boldsymbol{H}\right) \\ &= \sum_{s_{ij}\in\mathcal{S}} w_{ij} \log P\left(s_{ij}|\boldsymbol{h}_i, \boldsymbol{h}_j\right) + \sum_{i=1}^{N} \log P\left(\boldsymbol{h}_i\right)\end{aligned} \tag{1}$$

$$w_{ij} = \begin{cases} \left|\mathcal{S}\right| / \left|\mathcal{S}_1\right|, & s_{ij} = 1 \\ \left|\mathcal{S}\right| / \left|\mathcal{S}_0\right|, & s_{ij} = 0 \end{cases} \tag{2}$$

$$\begin{aligned}P\left(s_{ij}|\boldsymbol{h}_i, \boldsymbol{h}_j\right) &= \begin{cases} \sigma\left(\mathrm{d}\left(\boldsymbol{h}_i, \boldsymbol{h}_j\right)\right), & s_{ij} = 1 \\ 1 - \sigma\left(\mathrm{d}\left(\boldsymbol{h}_i, \boldsymbol{h}_j\right)\right), & s_{ij} = 0 \end{cases} \\ &= \sigma(\mathrm{d}\left(\boldsymbol{h}_i, \boldsymbol{h}_j\right))^{s_{ij}} \left(1 - \sigma\left(\mathrm{d}\left(\boldsymbol{h}_i, \boldsymbol{h}_j\right)\right)\right)^{1-s_{ij}}\end{aligned} \tag{3}$$

$$\sigma\left(\mathrm{d}\left(\boldsymbol{h}_i, \boldsymbol{h}_j\right)\right) = \frac{\gamma}{\gamma + \mathrm{d}\left(\boldsymbol{h}_i, \boldsymbol{h}_j\right)}, \tag{4}$$

$$P\left(\boldsymbol{h}_i\right) = \frac{\gamma}{\gamma + \mathrm{d}\left(\left|\boldsymbol{h}_i\right|, 1\right)}, \tag{5}$$

$$\begin{aligned}\mathrm{d}\left(\boldsymbol{h}_i, \boldsymbol{h}_j\right) &= \frac{K}{4} \left\| \frac{\boldsymbol{h}_i}{\|\boldsymbol{h}_i\|} - \frac{\boldsymbol{h}_j}{\|\boldsymbol{h}_j\|} \right\|_2^2 \\ &= \frac{K}{2} \left(1 - \cos\left(\boldsymbol{h}_i, \boldsymbol{h}_j\right)\right).\end{aligned} \tag{6}$$

$$\min_{\Theta} L + \lambda Q, \tag{7}$$

$$L = \sum_{s_{ij}\in\mathcal{S}} w_{ij} \left(s_{ij} \log \frac{\mathrm{d}\left(\boldsymbol{h}_i, \boldsymbol{h}_j\right)}{\gamma} + \log\left(1 + \frac{\gamma}{\mathrm{d}\left(\boldsymbol{h}_i, \boldsymbol{h}_j\right)}\right)\right), \tag{8}$$

$$Q = \sum_{i=1}^{N} \log\left(1 + \frac{\mathrm{d}\left(|h_i|, 1\right)}{\gamma}\right), \qquad (9)$$

2018 IJCAI Deep Joint Semantic-Embedding Hashing

Ning Li, Chao Li, Cheng Deng, Xianglong Liu, Xinbo Gao (with tensorflow code)

之前方法的缺点：()

Even so, **how to sufficiently discover the semantic correlation from label information is still a crucial issue to be addressed.**

Existing supervised hashing methods **do not rationally exploit label information of samples, almost all of which only simply construct the similarity affinity matrix of sample pairs.**

Each sample is annotated by single label, **where the similarity relationship between samples is very sparse, i.e., the number of similar pairs is much smaller than the number of dissimilar pairs, which will result in that the learned hash codes cannot preserve the original similarity relationship effectively.**

Unfortunately, **multiple labels in current methods are oversimplified to single-label case, which removes many useful semantic information and cannot maintain the original similarity relationship of sample pairs.**

However, **the data imbalance remains a challenge and almost all of these methods do not or little exploits semantic information to generate semantic correlation from label information directly.**

DSEH方法的优势：

We mainly focus on **extracting abundant semantic correlation from label information with deep neural network.**

Different from generating supervised information **only in the Hamming space** in most exiting methods, **a new semantic space is constructed in our method, with which similarity relationship can be well preserved at semantic level.**

可学习的例句：

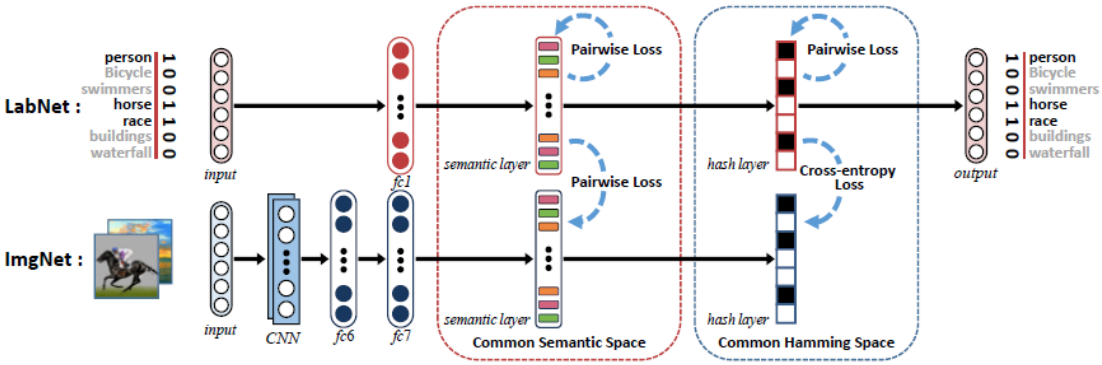


Figure 2: The framework of our proposed DSEH.

LabNet is an end-to-end fully connected deep neural network, where a semantic layer and a hash layer are built to **generate semantic features and hash codes from label information.** ImgNet consists of a convolution neural network with a semantic

layer and a hash layer, which is used to learn hash codes of the input images.

$$\min_{F^l,\theta^l} \mathcal{J}_1 = -\log p\left(\mathcal{S}|\,F^l\right) = -\sum_{s_{ij}\in\mathcal{S}} \left(s_{ij}\boldsymbol{f}_i^{l\top}\boldsymbol{f}_j^l - \log(1+\exp(\boldsymbol{f}_i^{l\top}\boldsymbol{f}_j^l))\right) \tag{5}$$

$$\min_{H^l,\theta^l} \mathcal{J}_2 = -\log p\left(\mathcal{S}|\,H^l\right) = -\sum_{s_{ij}\in\mathcal{S}} \left(s_{ij}\boldsymbol{h}_i^{l\top}\boldsymbol{h}_j^l - \log(1+\exp(\boldsymbol{h}_i^{l\top}\boldsymbol{h}_j^l))\right) \tag{7}$$

$$\min_{H^l,\theta^l} \mathcal{J}_3 = \sum_{s_{ij}\in\mathcal{S}} \left(\||\boldsymbol{h}_i^l| - \mathbf{1}\|_1 + \||\boldsymbol{h}_j^l| - \mathbf{1}\|_1\right) \tag{8}$$

$$\min_{\hat{\boldsymbol{Y}}^l,\theta^l} \mathcal{J}_4 = \sum_{i=1}^{N} \|\boldsymbol{y}_i^l - \hat{\boldsymbol{y}}_i^l\|_2^2 = \sum_{i=1}^{N} \|\boldsymbol{y}_i^l - \boldsymbol{w}^\top\boldsymbol{h}_i^l - \boldsymbol{b}\|_2^2 \tag{10}$$

$$\min_{F^l,H^l,\theta^l} \mathcal{L}_{\text{Lab}} = \mathcal{J}_1 + \alpha\mathcal{J}_2 + \beta\mathcal{J}_3 + \gamma\mathcal{J}_4 \tag{11}$$

$$\min_{F^v,\theta^v} \mathcal{J}_1 = -\log p\left(\mathcal{S}|\,F^v\right) = -\sum_{s_{ij}\in\mathcal{S}} \left(s_{ij}\boldsymbol{f}_i^{v\top}\boldsymbol{f}_j^l - \log(1+\exp(\boldsymbol{f}_i^{v\top}\boldsymbol{f}_j^l))\right) \tag{12}$$

$$\min_{H^v,\theta^v} \mathcal{J}_2 = -\sum_{i=1}^{N} \left[\boldsymbol{h}_i^{l'}\log\sigma(\hat{\boldsymbol{y}}_i^v) + (1-\boldsymbol{h}_i^{l'})\log(1-\sigma(\hat{\boldsymbol{y}}_i^v))\right] \tag{13}$$

$$\min_{F^v,H^v,\theta^v} \mathcal{L}_{\text{Img}} = \mathcal{J}_1 + \eta\mathcal{J}_2 \tag{14}$$

2018 AAAI Asymmetric Deep Supervised Hashing

Qing-Yuan Jiang, Wu-Jun Li (with MATLAB and Pytorch code)

之前方法的缺点：（）

However, most existing deep supervised hashing methods **adopt a symmetric strategy to learn one deep hash function for both query points and database (retrieval) points.** The **training** of these symmetric deep supervised hashing methods is **typically time-consuming**, which makes them **hard to effectively utilize the supervised information for cases with large-scale database.**

ADSH方法的优势：

ADSH treats the query points and database points in an asymmetric way. More specifically, **ADSH learns a deep hash function only for query points**, while the hash codes for **database points are directly learned.**

The training of ADSH is much more efficient than that of traditional symmetric deep supervised hashing methods.

To make the training practicable, most existing deep supervised hashing methods have to **sample only a small subset from the whole database to construct a training set for hash function learning, and many points in database may be discarded during training.** Hence, **it is hard for these deep supervised hashing methods to effectively utilize the supervised information for cases with large-scale database, which makes the search performance unsatisfactory.**

ADSH adopts a deep hash function to generate hash codes for query points, but **the binary hash codes for database points are directly learned.**

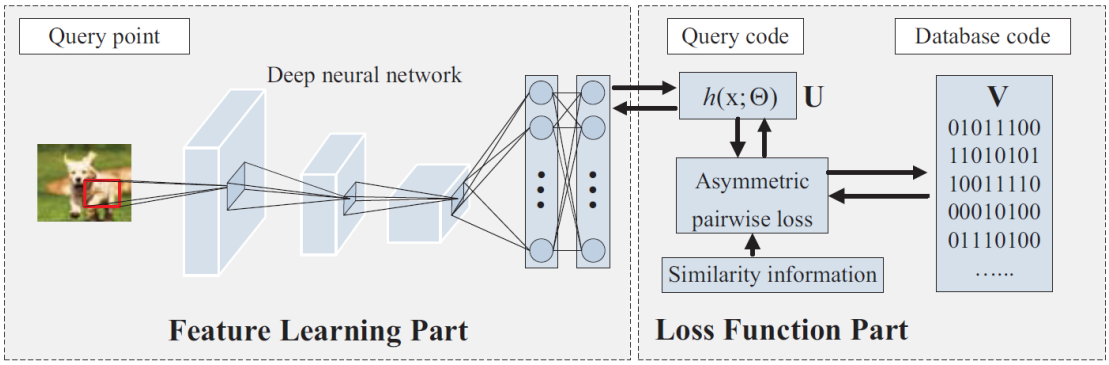
This asymmetric property of ADSH is different from traditional deep supervised hashing methods. **Traditional deep supervised hashing methods adopt the same deep neural network to perform feature learning for both query points and database points, and then use the same deep hash function to generate binary codes for both query points and database points.**

可学习的例句：

$$\min_{\Theta,\mathbf{V}} J(\Theta,\mathbf{V}) = \sum_{i=1}^{m}\sum_{j=1}^{n}\left[\tanh(F(\mathbf{x}_i;\Theta))^T\mathbf{v}_j - cS_{ij}\right]^2,$$
$$\text{s.t.} \quad \mathbf{V} \in \{-1,+1\}^{n\times c}, \tag{3}$$

$$\begin{aligned}\min_{\Theta,\mathbf{V}} J(\Theta,\mathbf{V}) = &\sum_{i\in\Omega}\sum_{j\in\Gamma}\left[\tanh(F(\mathbf{y}_i;\Theta))^T\mathbf{v}_j - cS_{ij}\right]^2 \\ &+\gamma\sum_{i\in\Omega}[\mathbf{v}_i - \tanh(F(\mathbf{y}_i;\Theta))]^2 \qquad (5)\\ \text{s.t.} \quad &\mathbf{V} \in \{-1,+1\}^{n\times c},\end{aligned}$$

$$\begin{aligned}\frac{\partial J}{\partial \mathbf{z}_i} = &\left\{2\sum_{j\in\Gamma}\left[(\tilde{\mathbf{u}}_i^T\mathbf{v}_j - cS_{ij})\mathbf{v}_j\right] + 2\gamma(\tilde{\mathbf{u}}_i - \mathbf{v}_i)\right\} \\ &\odot(1-\tilde{\mathbf{u}}_i^2). \qquad (6)\end{aligned}$$



用了整个数据集进行训练，网络只用于少量的查询样本，整个训练样本的二值码直接优化得到。解决了深度方法只能训练一部分数据的问题。想法很好。

2019

2019 CVPR K-Nearest Neighbors Hashing

Xiangyu He, Peisong Wang, Jian Cheng (with code)

之前方法的缺点：（）

BSH方法的优势：

可学习的例句：

Dual Asymmetric Deep Hashing Learning

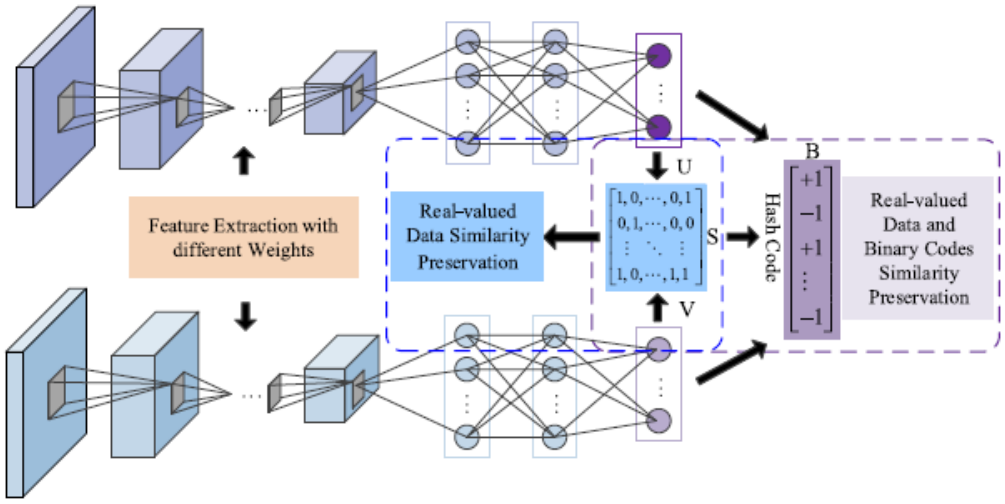


FIGURE 1. The framework of the proposed method. Two streams with five convolution layers and three full-connected layers are used for feature extraction. For the real-valued outputs from these two neural networks, their similarity is preserved by using a pairwise loss. Based on the outputs, a consistent hash code is generated. Furthermore, an asymmetric loss is introduced to exploit the semantic information between the binary code and real-valued data.

$$
\begin{aligned}
\min_{\mathbf{F},\mathbf{G},\mathbf{B}} L = & \left\|\tanh(\mathbf{F})\mathbf{B}^T - k\mathbf{S}\right\|_F^2 + \left\|\tanh(\mathbf{G})\mathbf{B}^T - k\mathbf{S}\right\|_F^2 \\
& - \tau \sum_{i,j=1}^{n} (S_{ij}\Theta_{ij} - \log(1 + e^{\Theta_{ij}})) \\
& + \gamma(\|\tanh(\mathbf{F}) - \mathbf{B}\|_F^2 + \|\tanh(\mathbf{G}) - \mathbf{B}\|_F^2) \\
& + \eta(\left\|\tanh(\mathbf{F})^T \mathbf{1}\right\|_F^2 + \left\|\tanh(\mathbf{G})^T \mathbf{1}\right\|_F^2) \\
\text{s.t. } & \mathbf{B} \in \{-1, +1\} \qquad (6)
\end{aligned}
$$

Unsupervised Deep Hashing with Similarity-Adaptive and Discrete Optimization

Zero-Shot Sketch-Image Hashing

Hashing as Tie-Aware Learning to Rank

IMPROVING BACK-PROPAGATION BY ADDING AN

ADVERSARIAL GRADIENT

Asymmetric Distances for Binary Embeddings

Asymmetric Deep Supervised Hashing

Qing-Yuan

Zero-shot hashing via transferring supervised knowledge

A Survey on Learning to Hash

Deep cross-modal hashing

Efficient training of very deep neural networks for supervised hashing

Deep semantic ranking based hashing for multi-label image retrieval

Deep Cross-Modal Hashing

Deep Semantic Ranking Based Hashing for Multi-Label Image Retrieval

Efficient training of very deep neural networks for supervised hashing

A Revisit of Hashing Algorithms for Approximate Nearest Neighbor Search

A Revisit on Deep Hashings for Large-scale Content Based Image Retrieval

Learning binary codes for maximum inner product search

Zero-shot hashing via transferring supervised knowledge

Supervised Hashing Using Graph Cuts and Boosted Decision Trees

Deep Supervised Hashing with Anchor Graph